



www.parallax.com/P2 sales@parallax.com support@parallax.com +1 888-512-1024

Parallax Propeller 2 (P2X8C4M64P)数据表

Propeller 2 是一款用于嵌入式系统的多核微控制器,可提供高速并行处理
体积小,电流消耗低。Propeller 的多个处理器可完全控制 I/O
引脚,改变时钟速度的灵活性、随意启动和停止的能力以及执行的能力
以独立或合作的方式同时完成任务。

Propeller 2 P2X8C4M64P 微控制器由 8 个相同的 32 位处理器 (称为 *cogs*)组成,每个处理器都具有
它们各自拥有独立的 RAM,并连接到一个公共集线器。该集线器提供 512 KB 的共享 RAM、一个 CORDIC 数学求解器,
和家政设施。该架构支持 64 个智能 I/O 引脚,每个引脚都能够实现许多自主
模拟和数字功能。Propeller 2 的汇编语言 (PASM2) 具有每条指令条件
执行、特殊循环机制和基于模式的指令跳过,以促进快速、紧凑的代码。

零件编号图例			
P2X	8 摄氏度	4 米	64P
螺旋桨 2	8 个齿轮 (处理器)	4 Mbit 集线器 RAM (512 KB)	64 个智能 I/O 引脚

有三个内存区域:寄存器 RAM、查找 RAM 和集线器 RAM。每个 cog 都有自己的寄存器 RAM。
和 Lookup RAM (统称为 Cog RAM) ,而 Hub RAM 由所有 cog 共享。

Propeller 2 (P2X8C4M64P)RAM内存配置				
地区	深度	宽度	程序计数器 地址范围 (十六进制)	PASM 指令 D/S 地址范围 (十六进制)
Cog “注册”RAM	512	32 位	\$00000..\$001FF	\$000..\$1FF
Cog “查找”RAM	512	32 位	\$00200..\$003FF	\$000..\$1FF
集线器内存	524,288	8 位	\$00400..\$7FFFF	\$00000..\$7FFFF

特征

八个强大的 32 位处理器,每个处理器均具有:

- 访问所有 I/O 引脚,以及四个快速 DAC 输出通道和四个快速 ADC 输入通道
- 512 个双端口寄存器 RAM,用于存储代码和快速变量
- 512 个双端口查找 RAM,用于代码、流查找和变量
- 能够直接从寄存器 RAM、查找 RAM 和集线器 RAM 执行代码
- 约 350 条用于数学、逻辑、计时和控制操作的独特指令
- 所有数学和逻辑指令 (包括 16 x 16 乘法)均在 2 个时钟周期内执行

- 用于解释型语言的 6 时钟自定义字节码执行器 · 能够将 Hub RAM 和/或 Lookup RAM 传输到 DAC 和引脚或 HDMI 调制器 · 能够将引脚和/或 ADC 传输到 Hub RAM · 使用具有 8 位有符号/无符号系数的 3 x 3 矩阵进行实时色彩空间转换 · 用于 8:8:8:8 数据的像素混合指令 · 16 个可轮询和等待的唯一事件跟踪器 · 3 个在可选事件上触发的优先中断 · 齿轮到齿轮注意信号,可实现快速协调 · 用于单步执行、断点和轮询的隐藏调试中断 · 8 级硬件堆栈,可实现最快的子程序调用/返回和推送/弹出操作 · 进位和零标志

中央枢纽为处理器提供以下服务:

- 20 位地址空间中的 512 KB 连续 RAM · 所有 cog 同时以每时钟 32 位顺序读/写 · 可以小端格式读取和写入字节、字或长整型数据 · 最后 16 KB RAM 可写保护
- 具有比例因子校正的 32 位流水线 CORDIC 求解器 · 32 位 x 32 位无符号乘法,结果为 64 位 · 64 位 / 32 位无符号除法,商为 32 位,余数为 32 位 · 64 位 → 32 位平方根 · 将 (X32, Y32) 旋转 Theta32 → (X32, Y32) · (Rho32, Theta32) → (X32, Y32) 极坐标转笛卡尔坐标 · (X32, Y32) → (Rho32, Theta32) 笛卡尔坐标转极坐标 · 32 → 5.27 无符号数转对数 · 5.27 → 32 对数转无符号数 · Cogs 可以每 8 个时钟启动一次 CORDIC 运算,并在 55 个时钟之后获得结果
- 16 个信号量位,具有原子读-修改-写操作 · 64 位自由运行计数器,每个时钟递增,复位时清除 · 高质量伪随机数生成器 (Xoroshiro128**) ,启动时进行真随机播种,更新每个时钟,为每个齿轮和引脚提供唯一的数据 · 启动、轮询和停止齿轮的机制 · 16KB 启动 ROM · 启动时加载到最后 16 KB 的 Hub RAM · SPI 加载器,用于从 8 针闪存或 SD 卡自动启动 · 串行加载器,用于从主机启动 ■ 十六进制和 Base64 下载协议 ■ 交互式终端 P2 监视器 ■ 交互式终端 TAQOZ Forth

64 个智能 I/O 引脚,每个引脚具有:

- 8 位、120 欧姆 (3ns)和 1k 欧姆 DAC,具有 16 位过采样、噪声和高/低数字模式 · 具有 5 个范围、2 个源和 VIO/GIO 校准的 Delta-sigma ADC · 几种 ADC 采样模式:自动 2n SINC2、可调 SINC2/SINC3、示波器 · 逻辑、施密特、引脚到引脚比较器和 8 位电平比较器输入模式 · 2/3/5/8 位一致输入滤波,采样率可选 · 合并来自相关引脚的输入, -3 到 +3 · 负或正本地反馈,带或不带时钟 · 以 4 个块为单位外部供电,以获得干净的模拟 Vdd 参考 · 高低输出的独立驱动模式:逻辑 / 1.5 k / 15 k / 150 k / 1 mA / 100 μA / 10 μA / 漂浮

- 可编程32位时钟输出、转换输出、NCO/占空比输出
- 三角波/锯齿波/SMPS PWM 输出,16 位帧,带 16 位预分频器
- 采用 32 位计数器的正交解码,支持位置和速度模式
- 16 种不同的 32 位测量,涉及一个或两个信号
- USB 全速和低速 (通过奇数/偶数引脚对)
- 同步串行发送和接收,1 至 32 位,高达时钟/2 波特率
- 异步串行发送和接收,1 至 32 位,高达时钟/3 波特率

六种时钟模式,全部由软件控制,可在源之间无故障切换:

- 内部 20+ MHz RC 振荡器,标称值约为 24 MHz,用作初始时钟源
- 带有内部负载电容的晶体振荡器,适用于 7.5 pF/15 pF 晶体,可为 PLL 供电
- 时钟输入,可馈送PLL
- 分数 PLL,带有 1..64 晶体分频器 --> 1..1024 VCO 乘法器 --> 可选 (1..15)*2 VCO 后分频器
- 内部~20 kHz RC 振荡器,用于低功耗操作 (130 μ A)
- 时钟可以停止以达到最低功耗,直到复位 (100 μ A,由于泄漏)

电源要求:

- 核心:1.8 VDC,通过 VDD 引脚供电
- 智能 I/O 引脚:3.3 VDC,通过 VIO 引脚以 4 个为一组供电

物理特性:

- 封装类型:裸露焊盘TQFP-100
- 尺寸:14 x 14 毫米
- 工作温度范围:AEC-Q100 2 级 (-40 至 +221 °F, -40 至 +105 °C)
- 湿气敏感度等级 (MSL) 3 (168 小时)

目录

特征	1
前言	5
硬件	5
引脚说明	5
硬件连接	7
最少连接	7
外部晶体	8
复位开关	8
SPI闪存启动存储器	8
MicroSD 启动内存	9
双启动内存	10
手术	11
主机通信	11
P2监视器	11

塔克兹	12
系统组织	12
齿轮	十三
齿轮内存	十三
寄存器 RAM	十三
查找RAM	14
执行	14
启动和停止齿轮	15
齿轮注意力	15
中心	16
集线器内存	16
Cog-to-Hub RAM 接口	16
系统时钟	17
锁	19
CORDIC 求解器	20
智能 I/O 引脚	21
方向和状态	21
I/O引脚时序	21
引脚模式	22
I/O引脚电路	二十六
每个独特 I/O 引脚配置的等效原理图	二十七
智能模式	33
重启	三十五
PROPELLER 2 汇编语言 (PASM2) 简介	三十六
系统特性	四十七
绝对最大电气额定值	四十七
直流特性	四十七
交流特性	四十八
包装	49
变更日志	50
视差公司	50

前言:本数据表简

要描述了 Propeller 2 多核微控制器的概念、特性和硬件。它比单页规格书更能提供功能参考。

如需更多文档和资源（包括编程工具），请访问www.parallax.com/P2。您可以在“文档”部分找到[此数据表的最新版本以及可注释的 Google 文档版本链接](#)。此外，还提供了有关 Propeller 2 及其 Spin2 和 PASM2 语言的更深入参考资料链接，其中可能包含可注释的 Google 文档。

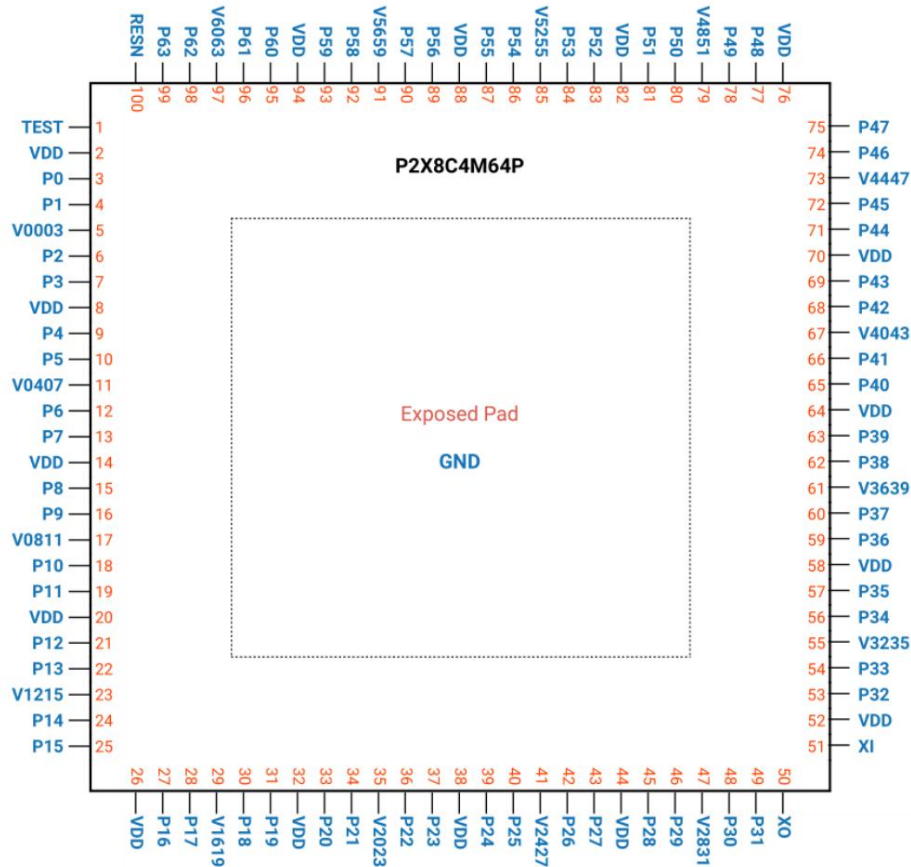
硬件:本文介绍

Propeller 2 微控制器的引脚布局和简化连接，仅供概念参考。对于大多数原型设计用途，Parallax 推荐使用预制 Propeller 2 电路，例如 P2 Edge 模块 (#P2-EC) 或 Propeller 2 评估板 (#64000)，其中包含所有推荐的连接和布局设计规则。请访问 Parallax 在线商店的 Propeller 2 专区，购买芯片、评估板、配件和开发者入门套件。

引脚说明

Propeller 2 (P2X8C4M64P)在其引脚排列方面具有显著的功率和散热考虑：

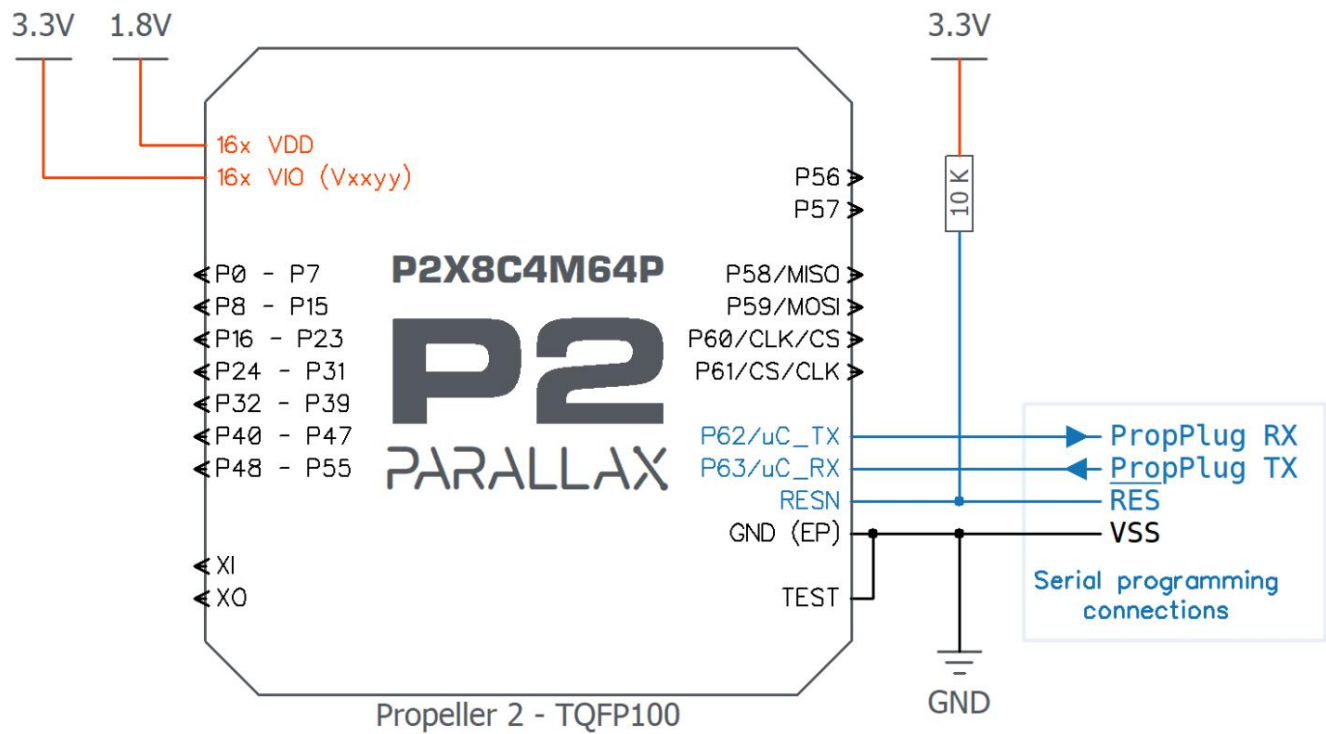
- 底部的大面积裸露焊盘既可作为核心和 I/O 引脚的接地参考 (GND)，又可作为冷却硅片的散热器
- 芯片周围均匀分布的多个电源引脚 (VDD)可确保整个
整个芯片
- 芯片周围均匀分布的多个 I/O 电源引脚 (Vxxyy) 为隔离、
干净的 DAC 和 ADC I/O 操作所需的超稳定电源参考



引脚说明			
引脚名称方向		V (典型值)	描述
接地	-	0	裸露焊盘 (芯片底面); 核心和智能引脚接地 - 内部连接至裸露焊盘。连接至地平面以便散热。
测试	.	0	接地
电源电压	-	1.8	核心力量
P0-63	输入/输出	0 至 3.3	智能引脚。P58-P63 在启动过程中发挥作用,之后用作通用用途。
维	-	3.3	为 4 个智能引脚供电:Pxx 至 Pyy
XO	哦	-	晶体输出。为外部晶体提供反馈,或者可以留空根据CLK寄存器设置断开。无需外部电阻或需要电容器。
+	.	-	晶体输入。可连接到晶体/振荡器组的输出 (带XO 断开),或连接到晶体的一条腿 (XO连接到晶体的另一条腿 晶振和谐振器,取决于 CLK 寄存器的设置。无需外部电阻或需要电容器。
可再生能源	.	0	重置 (低电平有效)。低电平有效时,重置螺旋桨:所有齿轮禁用,I/O 引脚悬空。RESn 从低电平变为高电平后 3 毫秒,Propeller 重新启动。连接一个电阻器以将其拉高至 3.3 V。

硬件连接

最少连接Propeller 2 通过四线
进行编程,并可选择包括外部晶体、复位开关、SPI Flash 和/或 microSD 内存。



最少的螺旋桨 2 连接

- 所有 VDD 引脚必须连接到单个 1.8 V 电源。· 所有 Vxxyy 引脚必须连接到 3.3 V。
- Vxxyy 引脚可以共用一个 3.3 V 电源,也可以分配到多个电源。通常,为模拟和数字功能供电的 Vxxyy 引脚会使用不同的电源。· 所有 VDD 和所有 Vxxyy 引脚必须具有靠近 GND 的旁路电容 (未显示)。
- 芯片下方的公共 GND (EP) 焊盘也用于散热。建议将 GND 焊盘通过多个过孔连接到实心接地层。
- TEST 引脚必须始终连接
- 到 GND。
- RESN (复位)引脚必须始终具有上拉电阻至 3.3 V (通常为 10 K Ω)。
- 使用串行接口 (例如 Parallax PropPlug #32201)进行编程和调试。
- 最小连接假设没有外部时钟源连接到 XI/XO 内部时钟必须是

使用 (调整源代码以匹配)。更多信息,请参阅[交流特性表](#)。

外部晶体内部时钟参考
适用于非常低功耗的应用,但精度也相当低。

对于需要更高精度的应用(例如:处理高速数据),外部时钟源连接到 XI/XO。

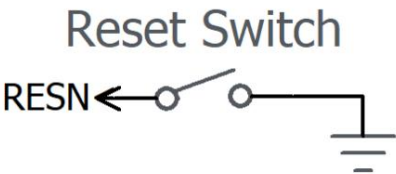
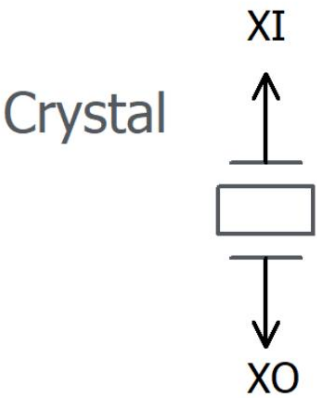
通常,晶体会连接在 XI 和 XO 之间,但外部时钟源也可以仅连接到 XI,常见选项包括时钟源发生器、振荡器或 MEMS 谐振器。

请参阅[交流特性](#)表以了解更多信息。

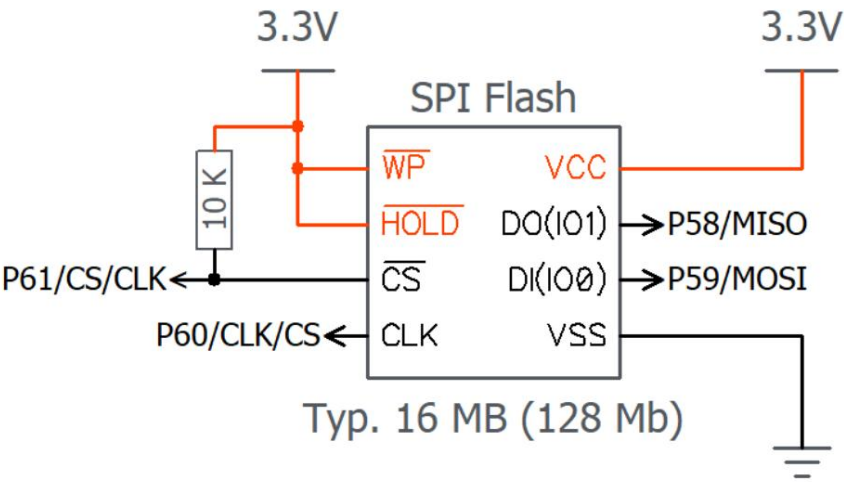
重置开关重置开关
是可选的,它是在开发期间或最终产品中重新启动 Propeller 2 的便捷方式。

该开关将 Propeller 2 复位引脚 (RESN) 驱动至地面,当保持低电平时,Propeller 2 保持处于休眠、低功耗状态。

请注意,必须始终使用外部电阻将 RESN 引脚拉高(至 3.3 V),如[最小连接](#)图所示。



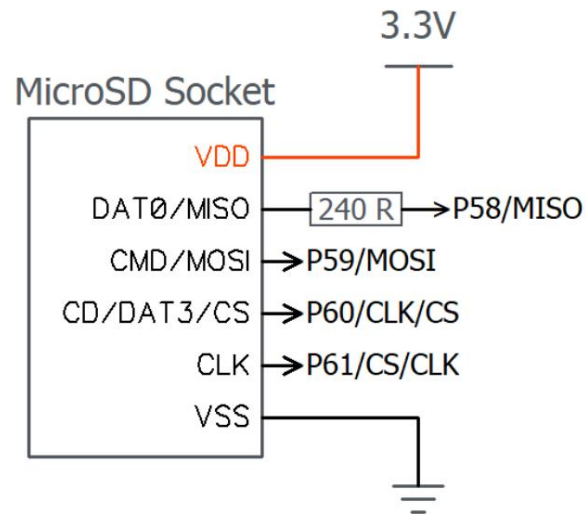
SPI闪存启动存储器



SPI Flash 启动存储器连接

- 请参阅所有关于[最小连接](#)的要求和建议。· 当 Propeller 2 使用此电路启动(或复位)时,将有一个 100 毫秒的串行编程窗口,然后自动从 SPI 闪存启动。如果 SPI 闪存启动失败,则会再经过 60 秒的串行编程窗口,然后关机。
- 每个 Propeller 2 固件映像最多需要 512 KB 空间。单个 SPI Flash 芯片可以容纳多个固件映像或代码片段,和/或用于用户数据。SPI Flash 芯片可作为通用内存区域供用户程序使用。

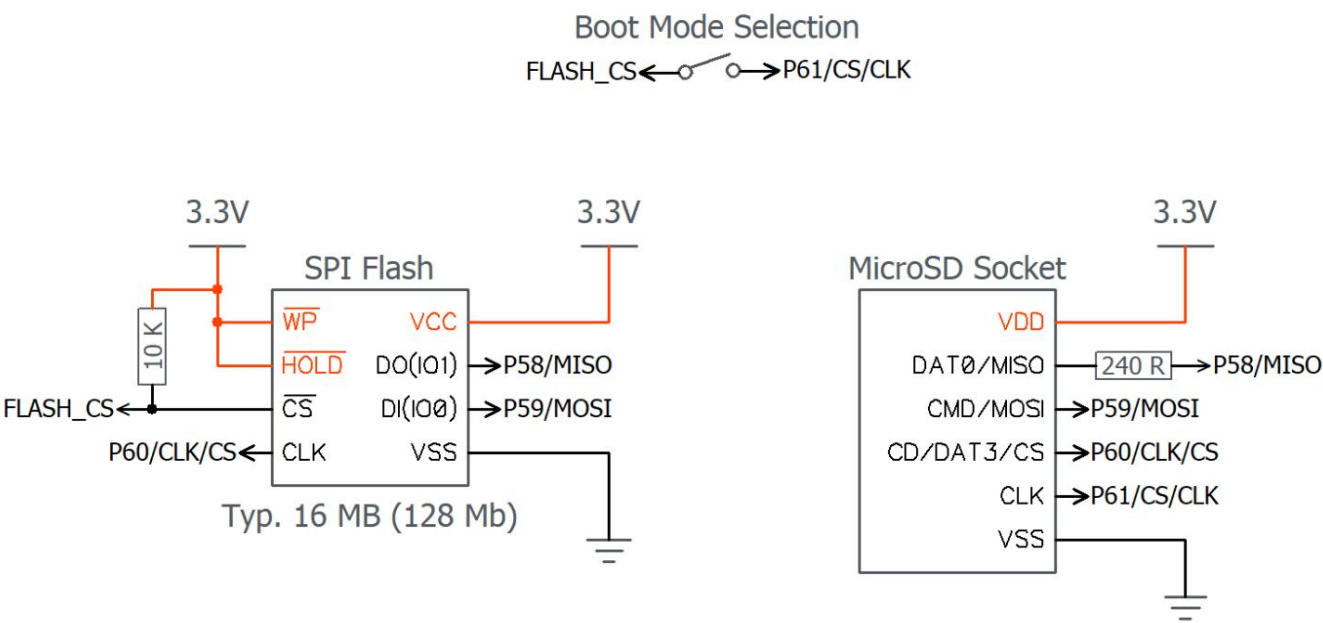
MicroSD 启动内存



MicroSD 启动内存连接

- 请参阅所有关于[最小连接](#)的要求和建议。· 当 Propeller 2 使用此电路启动（或重置）时，它将自动从 microSD 卡上保存的固件启动。如果 microSD 启动失败，则会在 60 秒的串行窗口后关机。
- microSD 卡必须格式化为 FAT32，并且启动固件文件必须保存在 microSD 卡的根目录中，文件名为：_BOOT_P2.BIX，或者如果未找到 .BIX 文件。
- 每个 Propeller 2 固件映像最多需要 512 KB。一张 microSD 卡可以容纳多个固件映像或代码片段，和/或用于用户数据。microSD 卡可作为通用存储区域供用户程序完全使用。

双启动内存



双启动内存连接

- 请参阅所有关于[最小连接](#)的要求和建议。· 启动模式选择开关决定活动的启动设备: [SPI 闪存](#)或 [microSD](#)。· [开/关闭](#)
合 (开启)= SPI 闪存启动模式: 当 Propeller 2 使用此电路启动 (或重置) 时, 将有一个 100 毫秒的串行编程窗口, 然后自动从存储在 SPI 闪存中的固件启动。如果 SPI 闪存启动失败, 则将再经过 60 秒的串行编程窗口, 然后关机。
- 开关打开 (关闭)= microSD 启动模式: 当 Propeller 2 使用此电路启动 (或重置) 时, 它将自动从 microSD 卡上保存的固件启动。如果 microSD 启动失败, 则将再等待 60 秒的串行窗口, 然后关机。

手术

假设电源稳定且 RESn 引脚“高”，Propeller 2 将使用以下步骤启动：

1. 在 5 毫秒内，Propeller 2 将其 Bootloader 加载到 cog 0 中。
2. 引导加载程序检查引脚 P59-P61 上的启动模式（配置），并通过串行与主机通信，或者从连接的闪存芯片或 SD 卡加载（启动）应用程序。
3. 所有进一步的活动都由应用程序本身定义。应用程序，以及开发者，完全控制修改内部时钟速度、I/O 引脚使用和行为、任何给定时间运行的齿轮等等。
4. 螺旋桨继续运转，直到所有齿轮相互关闭（或自行关闭），RESn 引脚低，或者应用程序请求重新启动。

请参阅 Propeller 2 硬件手册的启动程序部分以了解更多信息。

主机通信在典型操作中（如上图所示），

Propeller 2 将使用用户预先编写的 Propeller 应用程序启动；然而，同样的过程也允许加载新的应用程序或与内置系统交互。大多数启动模式（引脚 P59-P61）会触发一个串行通信窗口，主机可以通过引脚 P62 和 P63 与 Propeller 2 进行串行通信。

要从主机进入交互模式：

- 运行串行终端软件（如 Parallax 串行终端、TeraTerm 或 RealTerm）
- 禁用字符回显（Parallax 串行终端中的“Echo On”）
- 配置为 9600 波特到 2 MBaud（推荐）、8 个数据位、1 个停止位、无奇偶校验
- 按下并释放 Propeller 2 开发板的重置按钮
- 键入 > （大于号后跟一个空格），然后按 Ctrl+D 或 ESC 键进入 P2 监视器或

TAQOZ 模式，分别

P2 监视器

P2 监视器是一个内置的交互式系统，可用于查看和操作内存以及运行代码。使用 P2 监视器可以浏览和更改当前 RAM 内容，或从 microSD 内存加载和运行代码。开机或复位后（同时防止闪存/microSD 内存中驻留的应用程序自动运行），在终端中输入“>”（大于号后跟一个空格），然后按 Ctrl+D 键即可调用 P2 监视器。

下面是通过输入“000-010L”列出寄存器 RAM 的前 16 个长整数（长格式）的示例：

```
*000-010L
000:FF800800 FC0C003F F606C832 FCDC041F .....? ...2....
004:FD747E40 F0A6CA01 F426CA1F FD62CA00 .t-@.....&...b..
008:FB6EC9FA FC0C003F FD64C428 FF0007E0 .n.....?.d{....
00C:FB06012C FD655229 FF0007E1 FB0420B8 ...,eR).....
```

以下是 Hub RAM 的前 16 个字节的列表（长格式），输入“0000-0010L”：

```
*0000-0010L
00000:FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFF .....
```

有关更多信息，请参阅 Propeller 2 文档页面 (www.parallax.com/p2) 上的 [P2 Monitor](#) 链接。

要在 P2 监视器中切换到 TAQOZ，请键入 ESC，然后按 Enter 键。

TAQOZ

TAQOZ 是一款内置的交互式 Forth 语言引擎,基于 Tachyon Forth。使用 TAQOZ 可以探索“假设”,并快速运行 P2 硬件进行测试或调试。开机或复位后(同时防止闪存/microSD 卡中驻留的应用程序自动运行),在终端中输入“>”即可调用 TAQOZ。

ESC (大于号后跟一个空格),

然后按 Esc 键。

输入以下命令切换引脚 56 (例如:使 P56 上的 LED 闪烁):

五十六 眨眼

(输入“56 mute”停止切换)

...或者输入:

开始 56 高 250 毫秒 56 低 250 毫秒键直到 (按任意键停止切换)

有关更多信息,请参阅 Propeller 2 文档页面 (www.parallax.com/p2) 上的 TAQOZ 链接。

要在 TAQOZ 中切换到 P2 监视器,请键入 Ctrl+D。

系统组织Propeller 2 包括以下子系统。

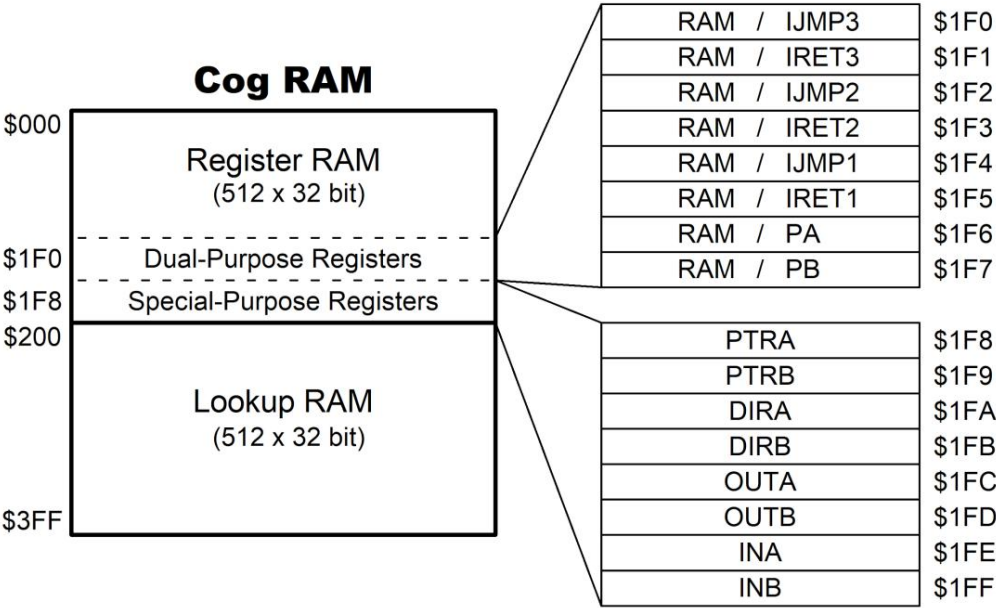
- Cogs (处理器) - 独立的 32 位处理单元 · 寄存器 RAM - cog 的私有内存,用于执行代码和快速操作数据 · 查找 RAM - cog 的半私有内存,用于执行代码和操作数据表/流 · Hub - 独占共享资源的访问管理器 · Hub RAM - 所有 cog 的共享内存,用于执行代码和操作数据 · 系统时钟 - 所有内部组件的时钟源 · 锁 - 16 个信号量位,用于协调共享资源的独占访问 · CORDIC 求解器 - 用于数学函数的流水线计算器
- 智能 I/O 引脚 - 带有可选智能电路的 I/O 引脚,可实现自主功能

齿轮

Propeller 包含多个处理器,称为齿轮。每个齿轮都有自己的 RAM,可以独立启动、停止和执行指令。所有活动的齿轮共享相同的系统时钟、集线器 RAM 和 I/O 引脚。

Cog RAM每个

cog 的 RAM 由两个 512 长度 (512 x 32)的块组成,称为寄存器 RAM 和查找 RAM,其组织方式如下图所示。



请注意,\$1FE (INA)和\$1FF (INB)也分别分别是调试中断调用地址和返回地址。

寄存器 RAM每个

cog 的主要 512 x 32 位双端口寄存器 RAM (简称 Reg RAM)提供代码执行、快速直接寄存器访问和特殊用途。它以 long (4 字节)的形式读写,包含通用寄存器、双用途寄存器和专用寄存器。

通用寄存器RAM 寄存器 \$000 到

\$1EF 是用于代码和数据的通用寄存器。

双用途寄存器RAM 寄存器

\$1F0 到 \$1F7 既可以用作通用寄存器,也可以在启用相关功能的情况下用作专用寄存器。

地址	姓名	目的
\$1F0	RAM/IJMP3	INT3 的中断调用地址
\$1F1	内存/IRET3	INT3 中断返回地址
\$1F2	RAM/IJMP2	INT2 的中断调用地址
\$1F3	内存/IRET2	INT2 的中断返回地址
\$1F4	RAM/IJMP1	INT1 的中断调用地址
\$1F5	内存/IRET1	INT1 中断返回地址
\$1F6	内存/PA	CALLD-imm 返回、CALLPA 参数或 LOC 地址
\$1F7	内存/PB	CALLD-imm 返回、CALLPB 参数或 LOC 地址

特殊用途寄存器RAM 寄存器 \$1F8

至 \$1FF 提供对八个特殊用途函数的映射访问。通常,当指定 \$1F8 至 \$1FF 之间的地址时,PASM 指令会访问特殊用途寄存器,而不仅仅是底层 RAM。

地址	姓名	目的
\$1F8	局部放射治疗	指向集线器 RAM 的指针 A 指
\$1F9	PTRB	向集线器 RAM 的指针 B
\$1FA	迪拉	P31..P0 的输出使能 P63..P32 的输
1FB美元	DIRB	出使能 P31..P0 的输出状态 P63..P32
\$1FC	奥塔	的输出状态 P31..P0 的输入状态
1FD 美元	输出	P63..P32 的输入状态
\$1FE	INA1	
\$1FF	INB2	

1同时调试中断调用地址

2同时调试中断返回地址

查找 RAM每个 cog

的辅助 512 x 32 位双端口查找 RAM (简称 LUT RAM)以 long (4 字节)的形式进行读写。它可用于：

- 临时空间 · 流访问 · 字
- 节码执行查找表 · 智能引脚数据
- 源 · Paired-Cog 通信机制 · 代码执行

暂存空间与寄存器

RAM 不同,cog 在其大多数 PASM 指令中无法直接引用查找 RAM 位置。相反,必须分别使用 RDLUT 和 WRLUT 指令在查找 RAM 和寄存器 RAM 之间读取或写入所需位置。这与其他硬件架构使用“LOAD”和“STORE”指令的暂存存储相同。使用 RDLUT 和 WRLUT 指令时,查找 RAM 的位置 \$200..\$3FF 可寻址为 \$000..\$1FF。

成对 Cog 通信机制：ID 号仅相差 LSB 的相邻 Cog

(例如 Cog 0 和 1、2 和 3 等)允许另一个 Cog 通过其本地查找 RAM 写入其查找 RAM。这使得相邻 Cog 能够通过其查找 RAM 快速共享数据。使用 SETLUTS 指令启用/禁用此功能,并在必要时使用 SETSE1..4 进行握手。请注意,此相邻 Cog 访问是在查找 RAM 的第二个端口上实现的,该端口也由流处理器在 DDS/LUT 模式下使用;这些端口不应同时使用。

Execution

Cogs 采用五级流水线执行架构。当执行流水线满载时,每条 PASM2 指令实际上只需两个时钟周期即可执行。如果某条指令暂停执行更多时钟周期,则流水线中所有后续指令也会暂停执行。任何被条件取消的指令仍将在流水线中移动,不会暂停或执行。分支指令会导致流水线刷新,因此分支指令之后的第一条指令至少需要五个时钟周期。

Cog 的程序计数器 (PC) 使用 20 位地址;在 P2X8C4M64P 上,高位 (19 位及以上)为“无关”位,可提供高达 512 KB 的执行空间。根据 Cog PC 的值,指令将从其寄存器 RAM、查找 RAM 或集线器 RAM 中获取。

PASM2 执行区域			
PC 地址	指令来源	内存宽度	PC增量
\$00000..\$001FF	齿轮寄存器 RAM	32 位	1
\$00200..\$003FF	Cog 查找 RAM	32 位	1
\$00400..\$7FFFF	集线器内存	8 位	4

寄存器执行
当 PC 位于 \$00000 到 \$001FF 的范围内时,cog 会从 Cog Register RAM 中获取指令。这是称为“cog 执行”。分支到 cog 寄存器地址时无需特殊考虑。

查找执行
当 PC 位于 \$00200 到 \$003FF 范围内时,cog 会从 Cog Lookup RAM 中获取指令。这是称为“lut 执行”。分支到 cog 查找地址时无需特殊考虑。

枢纽执行
当 PC 位于 \$00400 至 \$7FFFF 范围内时,cog 从 Hub RAM 中获取指令。这被称为“集线器执行模式”。集线器执行涉及特殊注意事项。

- 1. PC 超出 \$003FF 后不会启动集线器执行（它只会回到 \$00000）;分支必须发生从寄存器或查找执行到集线器执行。
- 2. 分支到集线器地址至少需要 13 个时钟周期。如果分支到的指令是非长对齐,则需要一个额外的时钟周期。
- 3. 从 Hub RAM 执行时,cog 使用 FIFO 硬件来加速指令,以便指令流将可用于连续执行。这意味着 FIFO 不能用于任何其他操作。因此,在集线器执行期间不能使用这些指令:

```
RDFAST/WRFAST/FBLOCK
RF字节/RF字/RF长/RF无功/RF无功
WFBYTE/WFWORD/WFLONG
XINIT / XZERO / XCONT - 当流媒体模式使用 FIFO 时
```

无法执行来自集线器地址 \$00000 到 \$003FF 的代码,因为 cog 将读取如上所示,从 cog 的寄存器 RAM 或查找 RAM 获取指令。

启动和停止齿轮
任何齿轮都可以启动或停止其他齿轮,也可以重新启动或停止自身。每个齿轮都有一个唯一的 ID,可用于启动或停止它。也可以启动空闲的（已停止或从未启动过的）齿轮,而无需知道它们的 ID。这样,应用程序可以根据需要简单地启动空闲的齿轮,当这些齿轮通过停止自己或当被其他人阻止时,他们返回到空闲齿轮池,以便再次重新启动。

PASM2 代码可以使用 COGID 指令识别自己的 cog（或获取其他 cog 的运行状态）,可以启动可以使用 COGINIT 指令启动齿轮,也可以使用 COGSTOP 指令停止齿轮。在 COGINIT 指令之前使用 SETQ 指令设置目标 cog 的 PTR 寄存器为 32 位值;用于将新 cog 指向运行时数据或传递单次启动值。

齿轮注意力
每个 cog 都可以使用 COGATN 指令来请求其他 cog 的注意。D 中的一个或多个操作数的低 8 位可以设置为高 (1),以指示相应的 cog 或 cogs。对于每个高位,匹配的 cog 会收到 POLLATN / WAITATN / JATN / JNATN 和中断使用的“注意”事件。注意

所有齿轮的频闪输出通过“或”运算组合在一起,形成一组由 8 个频闪组成的复合组,每个齿轮从中接收其特定的频闪。

在预期用例中,接收关注请求的 cog 知道哪个 cog 正在触发它以及如何响应。如果多个 cog 可能请求单个 cog 的关注,则可能需要在 Hub RAM 中实现某种消息传递结构来区分请求。

中心

集线器 RAM 全

局可访问的集线器 RAM 可以小端格式以字节、字和长整型进行读写。集线器地址始终面向字节。集线器 RAM 中的字和长整型没有特殊的对齐规则。Cog 可以从任何集线器地址开始读写字节、字和长整型,也可以从任何从 \$400 开始的集线器地址执行 PASM2 指令（长整型）。

集线器 RAM 的最后 16 KB 通常可在其正常地址范围以及 \$FC000..\$FFFFFF 范围内寻址。这为 16 KB 内部 ROM 提供了稳定的地址空间（无论未来 Propeller 2 版本如何）,该 ROM 在启动时会被缓存到集线器 RAM 的最后 16 KB 中。cog 调试方案也使用了这个高 16 KB 映射。

最后 16 KB RAM 可以隐藏在其正常地址范围之外,并在 \$FC000..\$FFFFFF 处设置为只读。这对于使最后 16 KB RAM 像 ROM 一样持久化非常有用。这也是实现调试的方式,因为映射到 \$FC000..\$FFFFFF 的 RAM 在调试中断服务例程中执行代码时仍然可以写入,从而允许原本受保护的 RAM 用作调试器应用程序空间和调试中断的 cog 寄存器交换缓冲区。

Cog-to-Hub RAM 接口：Hub RAM 由

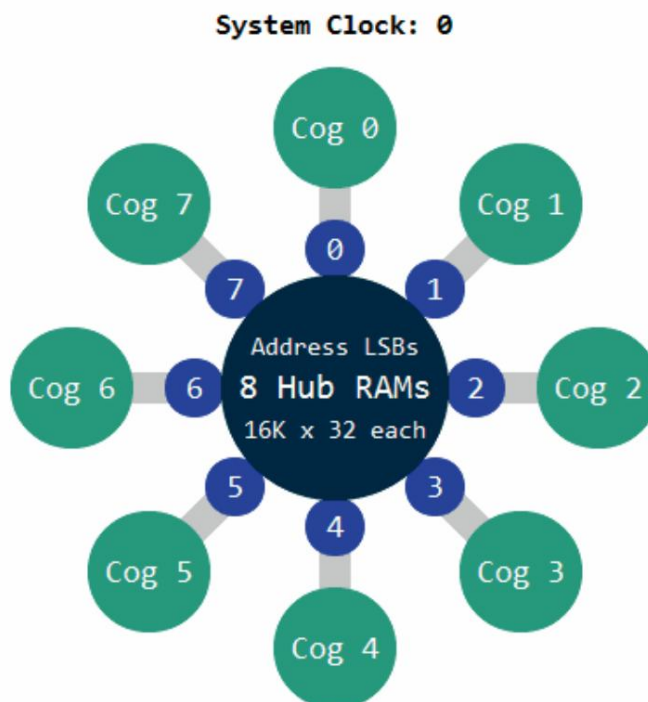
32 位宽的单端口 RAM 组成,具有字节级写入控制。该 RAM 被拆分成多个切片（每个 cog 一个）,并在所有 cog 之间进行复用。在 Propeller 2 (P2X8C4M64P) 上,每个 RAM 切片保存复合 Hub RAM 中每 8 个长整型值。在每个时钟周期,每个 cog 都可以访问“下一个”RAM 切片,从而实现连续的 Hub RAM 长整型值的双向传输。Hub RAM 接口图从概念上说明了这一过程:RAM 切片集合不断旋转,每个切片在每个时钟周期都会面向一个新的 cog。

当 cog 想要读取或写入 Hub RAM 时,它必须等待最多 7 个时钟周期才能访问所需的初始 RAM 片段。完成此操作后,之后每个时钟周期都可以访问后续位置（片段）,从而实现 32 位长度数据的连续读取或写入。

通常情况下,如果 cog 选择在下一个时钟周期不访问下一个可用位置,它必须再次等待最多 7 个时钟周期才能与所需的切片重新对齐。但是,每个 cog 都有一个可选的 hub FIFO 接口,可以平滑数据流,以实现低于 32 位/时钟周期的访问。此 hub FIFO 接口可设置为 hub RAM 读取或 hub RAM 写入操作,从而允许 hub RAM 以任意字节、字或长整型的组合顺序读取或顺序写入,速率最高为每时钟周期一个长整型。无论传输频率或字长如何,FIFO 都能确保 cog 的读取或写入操作均能正确地/向复合 hub RAM 进行。

P2 Hub RAM Interface

Every cog can read/write 32 bits per clock



Cogs 可以通过顺序 FIFO 接口访问 Hub RAM,也可以等待所需的 RAM 片段,同时让位于 FIFO。如果 FIFO 不忙（如果没有数据读取或写入,很快就会忙）,随机访问将有充分的机会访问复合 Hub RAM。

集线器 FIFO 接口有三种使用方式,但每次只能使用其中一种:

- 集线器执行（当 PC 为 \$00400..\$FFFFFF 时）
- 流媒体使用（从集线器 RAM → 引脚/DAC,或从引脚/ADC → 集线器 RAM 的后台传输）
- 软件使用（快速顺序读取或顺序写入指令）

对于流处理器或软件使用,FIFO 操作必须通过从 Cog RAM（寄存器/查找,\$000000..\$003FF）执行的 RDFAST 或 WRFAST 指令来建立。之后,在 Cog RAM 中,可以启用流处理器在后台开始移动数据,或者使用双时钟 RFxxxx/WFxxxx 指令手动读写顺序数据。

FIFO 包含 (#cogs+11) 个阶段。在读取模式下,只要少于 (#cogs+7) 个阶段已填充,FIFO 就会持续加载,超过此值后,最多可再有 5 个 long 数据流入,从而可能填满所有阶段。

这些指标确保 FIFO 在所有潜在的读取场景下都不会下溢。

系统时钟系统时钟是

所有内部组件的时间基准,可以通过多种方式进行配置。

- 直接来自内部慢时钟（RCSLOW）;~20 kHz 振荡器用于低功耗操作
- 直接来自内部快速时钟（RCFAST）;20 MHz+ 振荡器设计用于最低 20 MHz 操作
- 直接来自 XI 引脚;通过时钟振荡器或晶体振荡器从外部驱动
- PLL 修改的 XI 引脚;通过时钟振荡器或晶体振荡器从外部驱动,信号由 PLL（锁相环）内部修改,通常为高出几倍的频率

系统时钟由正在运行的 Propeller 2 应用程序使用以下格式的HUBSET指令配置：

集线器 ###%0000_000E_DDDD_DDMM_MMMM_MMMM_PPPP_CCSS

设置时钟模式

下表描述了位字段（E、D、M、P、C 和 S）。

PLL设置	价值	影响	笔记
%E	0/1	PLL 关闭/打开	XI 输入必须通过 %CC 启用。允许 10ms 在切换到 PLL 时钟之前,晶体+PLL 保持稳定来源。
%DDDDDD	0..63	1..XI 引脚的 64 个分区 频率	该分频器 XI 频率馈入相位频率比较器的“参考”输入。
%MMMMMMMM	0..1023	1..1024 除法 VCO频率	该分频 VCO 频率馈入相位频率比较器的“反馈”输入。这分频具有乘以被分频后的 XI 频率（每个 %DDDDDD）在 VCO 内部。VCO 频率应保持在100MHz至200MHz之间。
%PPPP	0 1 2 3 4 5 6 7 8 9 10 11 12 +三 14 15	压控振荡器 / 2 压控振荡器 / 4 压控振荡器 / 6 压控振荡器 / 8 纯椰子油 / 10 原精/12 压克力 / 14 压控振荡器 / 16 原精/18 纯椰子油 / 20 原精/22 压克力 / 24 压克力 / 26 压克力 / 28 纯椰子油 / 30 压控振荡器 / 1	该分频 VCO 频率可选择为系统当 SS = %11 时时钟。

%CC	XI 状态	XO 状态	XI/XO阻抗	XI/XO 装料帽
%00	被忽略	漂浮	高阻抗	离开
%01	输入	600欧姆驱动器	1兆欧姆	离开
%10	输入	600欧姆驱动器	1兆欧姆	每针 15pF
%11	输入	600欧姆驱动器	1兆欧姆	每针 30pF

%SS 时钟源	注释	
%11	锁相环	CC != %00 和 E=1,在切换到 PLL 之前,留出 10ms 的时间让晶体+PLL 稳定下来
%10	+-	CC != %00,等待 5ms 晶振稳定后再切换到 XI 引脚
%01	RCSLOW ~20 kHz,	可随时切换,低功耗
%00	远程快速连接	可以随时切换到 20+ MHz（名义上~24 MHz）在启动时使用。

警告 :错误地切换 PLL 设置 (%SS = %11) 可能会导致故障,从而挂起时钟电路。为了安全切换,请始终先使用 HUBSET 切换到内部振荡器 #F0（用于 RCFast）或 HUBSET #F1（用于 RCSLOW）。

PLL 示例PLL 将

XI 引脚频率从 1 到 64 分频,然后在 VCO 中将所得频率从 1 到 1024 倍增。VCO 频率可以直接使用,也可以除以 2、4、6、...30,得到最终的 PLL 时钟频率,用作系统时钟。

PLL 的 VCO 设计运行频率在 100 MHz 至 200 MHz 之间,应保持在该范围内。

$$\begin{aligned} &= \frac{() \times (\% + 1)}{(\% + 1)} \\ &= (\% = (\% = 15) \Rightarrow \\ &\neq 15) \Rightarrow \frac{(\% + 1) \times 2}{(\% + 1)} \end{aligned}$$

假设您有一个 20 MHz 晶振连接到 XI 和 XO,并且您希望以 148.5 MHz 的频率运行 Prop2。您可以将晶振除以 40 (%DDDDDD = 39)以获得 500 kHz 的参考频率,然后在 VCO 中将其乘以 297 (%MMMMMMMMMM = 296)以获得 148.5 MHz。您可以将 %PPPP 设置为 %1111 以直接使用 VCO 输出。配置值为 %1_100111_0100101000_1111_10_11。最后两个 2 位字段用于选择 15 pf 晶振模式和 PLL。但是,为了实现此时钟设置,必须分几个步骤完成:

```
集线器 #SF0                                设置 20 MHz+ (RCFAST) 模式
HUBSET ##%1_100111_0100101000_1111_10_00  启用晶体+PLL,保持在 RCFAST 模式  等待约 10ms 以使晶体+PLL 稳定下来
等待 ##20_000_000/100
HUBSET ##%1_100111_0100101000_1111_10_11  现在切换到以 148.5 MHz 运行的 PLL
```

由 %SS 位控制的时钟选择器具有一个去毛刺电路,该电路会等待旧时钟源的正沿出现后才断开连接,并保持其输出为高电平,然后等待新时钟源的正沿出现后才切换到新时钟源。在等待晶振和/或 PLL 稳定运行之前,需要选择模式 %00 或 %01,然后再切换到其中一种模式。

锁

为了实现应用程序定义的 cog 协调,hub 提供了一个由 16 个信号量位组成的池,称为锁。例如,cog 可以使用锁来管理资源的独占访问,或表示在多个 cog 之间共享的独占状态。锁的具体含义完全取决于使用它的应用程序;它们是一种允许某个 cog 每次拥有特定锁 ID 的“所有者”独占状态的手段。为了发挥作用,所有参与 cog 必须就锁的 ID 及其用途达成一致。

LOCK 指令如下:

```
锁新          D {WC}
锁定RET {#}D
锁具 {#}D {WC}
锁定REL {#}D {WC}
```

锁的使用 :要使

用锁,一个 cog 必须首先使用 LOCKNEW 分配一个锁,并将该锁的 ID 告知其他协作 cog。然后,协作 cog 使用 LOCKTRY 和 LOCKREL 分别获取或释放该锁所代表状态的所有权。如果应用程序不再需要该锁,可以通过执行 LOCKRET 将其返回到未分配锁池。一个 cog 可以分配多个锁。

任何时候,一个齿轮都可以使用 LOCKTRY 尝试拥有一个锁 (即:该锁所代表的状态)。Hub 会授予或拒绝所有权,以确保任何时候最多只有一个齿轮拥有该锁。如果一个齿轮

获得所有权后,它可以执行为该锁定义的任务,然后使用 LOCKREL 释放所有权,允许任何其他 cog 尝试获得所有权。只有获得锁所有权的 cog 才能释放它;但是,如果所有者 cog 停止 (COGSTOP)或重新启动 (COGINIT),锁也会被隐式释放。

CORDIC 求解器

该中心包含一个 54 级流水线 CORDIC 求解器 (坐标旋转数字计算机),可以为所有齿轮计算以下函数:

·乘法: 32 x 32 无符号乘法,乘积为 64 位 除法: 64 / 32 无符号除法,商为 32 位,余数为 32 位 平方根: 64 位无符号值的根,结果为 32 位 旋转: 32 位有符号 (X, Y) 绕 (0, 0) 旋转 32 位角度,结果为 32 位有符号 (X, Y) 笛卡尔转极坐标: 32 位有符号 (X, Y) 转 32 位 (长度、角度) 笛卡尔转极坐标运算 极坐标转笛卡尔: 32 位 (长度、角度) 转 32 位有符号 (X, Y) 极坐标转笛卡尔运算 整数转对数: 32 位无符号整数转 5:27 位对数 对数转整数: 5:27 位对数转32位无符号整数



每个 cog 可以在其每个 hub 访问窗口 (每八个时钟周期发生一次)发出一条 CORDIC 指令,并在 55 个时钟周期后通过 GETQX 和 GETQY 指令检索结果。为了提高吞吐量,cog 可以利用 hub 访问窗口和 CORDIC 流水线关系,发出 CORDIC 指令流,并与检索相应的结果交错,从而实现每八个时钟周期最多一个 CORDIC 结果。每个 cog 的活跃 CORDIC 指令和即将得到的结果彼此完全隔离,并且与其他 cog 完全隔离;但是,每个结果必须按时检索,否则将被后续结果 (如果有)覆盖。

乘法使用

QMUL 指令将两个无符号的 32 位数字相乘,并使用 GETQX 和 GETQY 指令 (分别用于低位和高位长)检索 CORDIC 结果。

除法使

用 QDIV 或 QFRAC 指令 (可选择使用前面的 SETQ 指令)将 64 位分子除以 32 位分母,然后使用 GETQX 和 GETQY 指令 (分别用于商和余数)检索 CORDIC 结果。

平方根对 64 位数

字使用 QSQRT 指令,并使用 GETQX 指令检索平方根 CORDIC 结果。

旋转

使用 SETQ 指令,然后使用 QROTATE 指令将 32 位有符号 Y 和 X 点对旋转一个无符号 32 位角度,然后分别使用 GETQX 和 GETQY 指令针对 X 和 Y 检索 CORDIC 结果。

笛卡尔到极坐标使用

QVECTOR 指令将 (X, Y) 笛卡尔坐标转换为 (长度, 角度) 极坐标,并使用 GETQX 和 GETQY 指令 (分别用于长度和角度)检索 CORDIC 结果。

极坐标到笛卡尔坐标使用

QROTATE 指令将 (长度、角度) 极坐标转换为 (X, Y) 笛卡尔坐标,并使用 GETQX 和 GETQY 指令 (分别针对 X 和 Y)检索 CORDIC 结果。

整数到对数

对无符号 32 位整数使用 QLOG 指令并检索 5:27 位对数 CORDIC 结果（5 位使用 GETQX 指令获取 16 位整数（即指数和 27 位尾数）。

对数转整数

对 5:27 位对数使用 QEXP 指令,并使用 GETQX指令。

智能 I/O 引脚

每个 I/O 引脚都具有多功能数字和模拟功能以及自主状态机功能
否则将需要处理器时间来执行。引脚模式和智能模式的组合
为应用程序设计提供了熟练的功能,增加了 Propeller 2 的潜力,超越了多核架构本身就提供了。有24种低级引脚模式和34种高级智能模式。

每个 I/O 引脚的行为由四种设置的组合描述:1)方向（输入/输出）,2)状态（输出驱动/输入感应）、3)引脚模式和 4)智能模式（可选）。

方向和状态

最简单的形式是,I/O 引脚通过专用 cog 寄存器和影响它们的指令进行控制。

I/O引脚寄存器		
注册 Cog 地址	目的	
迪拉	\$1FA	P0..P31 的输出使能位（高电平有效）
DIRB	1FB美元	P32..P63 的输出使能位（高电平有效）
奥塔	\$1FC	P0..P31 的输出状态位（相应的 DIRA 位必须为高才能启用输出）
输出	1FD 美元	P32..P63 的输出状态位（相应的 DIRB 位必须为高才能启用输出）
伊纳	\$1FE	P0..P31 的输入状态位
异烟肼	\$1FF	P32..P63 的输入状态位

通用和特殊引脚指令可以写入DIRA/DIRB/OUTA/OUTB来影响引脚输入/输出行为,并可从 INA / INB 读取以检索引脚状态。通用指令可在整个 32 位寄存器（所有引脚）,而特殊引脚指令对其中的单个位（引脚)进行操作。

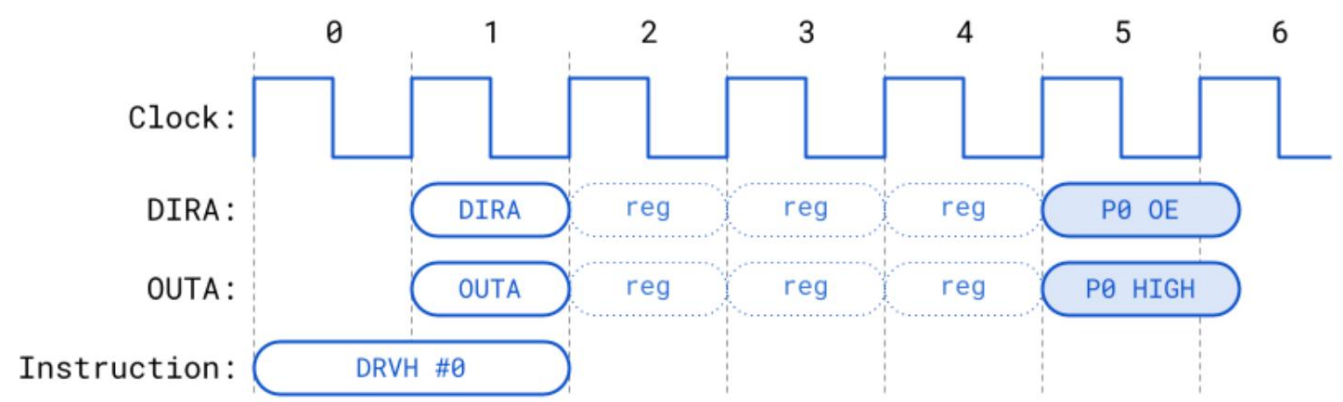
特殊密码说明	
指示	目的
DIRL/DIRH/DIRC/DIRNC/DIRZ/DIRNZ/DIRRND/DIRNOT {#}D	影响 DIRx 中的引脚 D 位
OUTL/OUTH/OUTC/OUTNC/OUTZ/OUTNZ/OUTRND/OUTNOT {#}D	影响 OUTx 中的引脚 D 位
FTL/FLTH/FLTC/FLTNC/FLTZ/FLTNZ/FLTRND/FLTNOT {#}D	影响 OUTx 中的引脚 D 位,清除 DIRx 中的位
DRVL/DRVH/DRVC/DRVNC/DRVZ/DRVNZ/DRVNRND/DRVNOT {#}D	影响 OUTx 中的引脚 D 位,设置 DIRx 中的位
测试 {#}D WC/WZ/ANDC/ANDZ/ORC/ORZ/XORC/XORZ	读取 INx 中的引脚 D 位并影响 C 或 Z
TESTPN {#}D WC/WZ/ANDC/ANDZ/ORC/ORZ/XORC/XORZ	读取 !INx 中的引脚 D 位并影响 C 或 Z

所选的引脚模式和智能模式（如果不是默认模式)可能会覆盖上述部分内容,如下所述
在稍后的相应章节中

I/O引脚时序

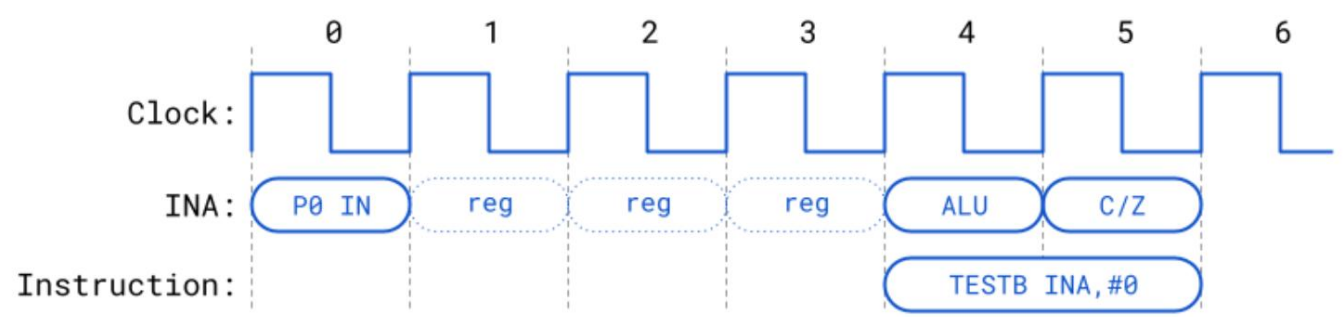
在每个物理 I/O 引脚和控制它们的齿轮之间,有一个由三个单位寄存器 (reg) 组成的链。
实时信号（输入或输出)在到达目的地的途中穿过该链,如下所述。

当任何指令更改 DIRx/OUTx 位时,引脚需要经过指令执行后三个额外的时钟周期才能开始转换到新状态。此处演示了此延迟,使用 DRVH 设置 I/O 引脚 P0 的输出使能 (OE) 并将 P0 的输出锁存器驱动至高电平。

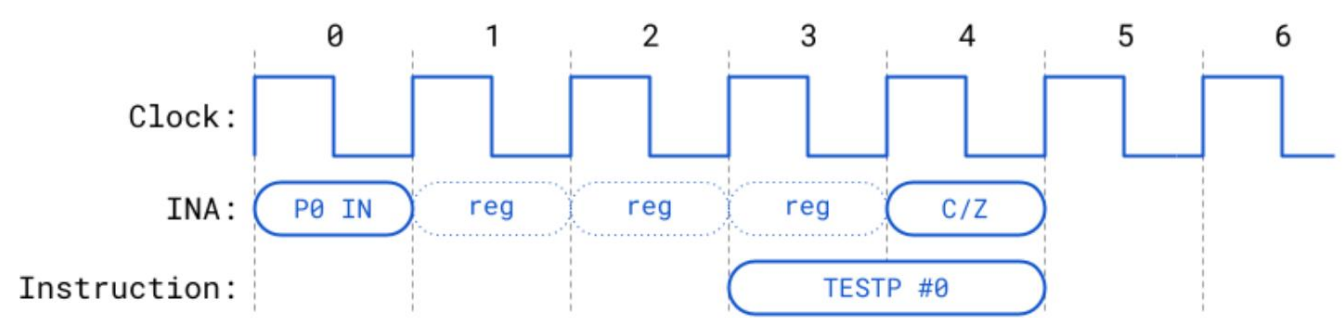


请注意,“P0 OE”和“P0 HIGH”在时钟 5 的上升沿开始转换;然而,转换完成所需的时间取决于时钟频率和电路负载。I/O 引脚是异步的(并非严格依赖于时钟),因此,如果工作频率较低,转换可能在 1 个时钟周期内完成;而如果工作频率较高,则可能需要多个时钟周期才能完成。

当指令读取 INx 寄存器时,它将反映指令开始前三个时钟周期内注册的引脚状态。以下使用 TESTB 演示了此延迟:



当使用 TESTP/TESTPN 指令读取引脚时,读取的值将反映该引脚在指令开始前两个时钟周期内注册的状态。实际上,TESTP/TESTPN 获取的 INx 数据比 INx 寄存器中可用的数据更新:



引脚模式每个 I/

O 引脚都有 13 个低电平引脚模式配置位,用于确定其 3.3 V 电路的工作模式 (24 个模式中的 1 个)。引脚模式使用 WRPIN 指令设置,该指令 D 操作数中的 13 个“M”位指定引脚模式配置。请注意,在某些智能引脚模式下,配置位会被部分覆盖,以设置 DAC 值等参数。

WRPIN 的 D 操作数值的格式为：

%AAAA_BBBB_FFF_MMMMMMMMMMMM_TT_SSSSS_0

- A = PIN 输入选择器 · B = ADJ 输入选择器 · F = PIN 和 ADJ 输入逻辑/过滤（应用于 PIN 和 ADJ 输入选择器的结果） · M = 引脚模式 · T = 引脚 DIR/OUT 控制（默认 = %00） · S = 智能模式

(A)PIN 或 (B)ADJ 输入选择器	
%AAAA %BBBB	选择
0xxx true	（默认）
1xxx 倒置	
x000	该引脚的读取状态（默认）
x001 相对 +1	引脚的读取状态
x010 相对+2	引脚的读取状态
x011 相对+3	引脚的读取状态
x100	该引脚的 OUT 位来自 cogs
x101 相对 -3	引脚的读取状态
x110 相对 -2	引脚的读取状态
x111 相对 -1	引脚的读取状态

(F)PIN 和 ADJ 逻辑/过滤	
%FFF 逻辑/过滤器	
000 A,B	（默认）
001 A 与 B	
010 A 或 B	
011 A 异或 B	
100 A,B,均使用全局 filt0 设置进行过滤	
101 A,B,均使用全局 filt1 设置进行过滤 110 A,B,均使用全局 filt2 设置进行过滤 A,B,均使用全局 filt3 设置进行过滤结果 “A”	
111	将在非智能引脚模式下驱动 IN 信号。

(M) 引脚模式									
WRPIN D[20:8]配置					最终的内部配置				
兆欧表[12:0]	输入	引脚输出1	CIOHHLLL OE2	DAC ADC	ADC 模式	比较器			
0000_CIOHHLLL	引脚逻辑	出去	哎呀哎呀	目录	0	0			0
0001_CIOHHLLL	引脚逻辑	输入	哎呀哎呀	目录	0	0			0
0010_CIOHHLLL	调整逻辑	输入	哎呀哎呀	目录	0	0			0
0011_CIOHHLLL	施密特	出去	哎呀哎呀	目录	0	0			0
0100_CIOHHLLL	施密特	输入	哎呀哎呀	目录	0	0			0
0101_CIOHHLLL	阿杰 施密特	输入	哎呀哎呀	目录	0	0			0
0110_CIOHHLLL	固定 > 调整	出去	哎呀哎呀	目录	0	0			固定 > 调整
0111_CIOHHLLL	固定 > 调整	输入	哎呀哎呀	目录	0	0			固定 > 调整
100000_哦哦哦	ADC,GND ADC,	出去	10 哦哦哦	目录	0	1	000		0
100001_哦哦哦	Vxxyy ADC,浮动	出去	10 哦哦哦	目录	0	1	001		0
100010_哦哦哦	ADC,引脚 1x	出去	10 哦哦哦	目录	0	1	010		0
100011_哦哦哦	ADC,引脚 3.16x	出去	10 哦哦哦	目录	0	1	011		0
100100_哦哦哦	ADC,引脚 10x ADC,引	出去	10 哦哦哦	目录	0	1	100		0
100101_哦哦哦	脚 31.6x ADC,引脚	出去	10 哦哦哦	目录	0	1	101		0
100110_哦哦哦	100x	出去	10 哦哦哦	目录	0	1	110		0
100111_哦哦哦		出去	10 哦哦哦	目录	0	1	111		0
10100_DDDDDDDD	ADC,引脚 1x ³	DAC 990 Ω,3.3 V DAC	10xxxxxxx	0	目录	出去	011		0
10101_DDDDDDDD	ADC,引脚 1x ³	600 Ω,2.0 V DAC 123.75	10xxxxxxx	0	目录	出去	011		0
10110_DDDDDDDD	ADC,引脚 1x ³	Ω,3.3 V DAC 75 Ω,2.0 V	10xxxxxxx	0	目录	出去	011		0
10111_DDDDDDDD	ADC,引脚 1x ³		10xxxxxxx	0	目录	出去	011		0
1100_CDDDDDDDD	引脚 > D	输出,1.5 kΩ !输	C00001001	目录	0	0			引脚 > D
1101_CDDDDDDDD	引脚 > D	入,1.5 kΩ 输入,1.5	C01001001	目录	0	0			引脚 > D
1110_CDDDDDDDD	调整 > D	kΩ !输入,1.5 kΩ	C00001001	目录	0	0			调整 > D
1111_CDDDDDDDD	调整 > D		C01001001	目录	0	0			调整 > D

1OUT 表示输出锁存位驱动输出;输入表示 “输入”列的项目驱动输出

2 OE 仅使能数字逻辑输出;模拟输出在 DAC 列中指示

3 如果 OUT 位 = 1

引脚模式图例									
C 输入/输出		哈哈	哈哈	哈哈	哈哈	驾驶			
0	居住 ¹					000	快速		
1	已计时 ²					001	1.5 kΩ		
我输入						010	15 kΩ		
0	真的					011	150 kΩ 1		
1	不 (反转)					100	mA		
O 输出						101	100 μA 10		
0	真的					110	μA 浮动		
1	不 (反转)					111			
OE = 数字输出使能 (当 DIR 位高时) DAC = 数模转换器使能 (当 DIR 位为高时) ADC = 模数转换器使能 (固定,或当 OUT 位为高时) OUT = 输出锁存位;0:低,1:高。 例外:DAC 模式使用 OUT 作为 0:禁用,1:启用。 DIR = 方向位;0:输入 (浮点),1:输出 (驱动) 例外:DAC 模式使用 DIR 为 0:禁用,1:启用。 DDDDDDDD 和 D = DAC 级别									

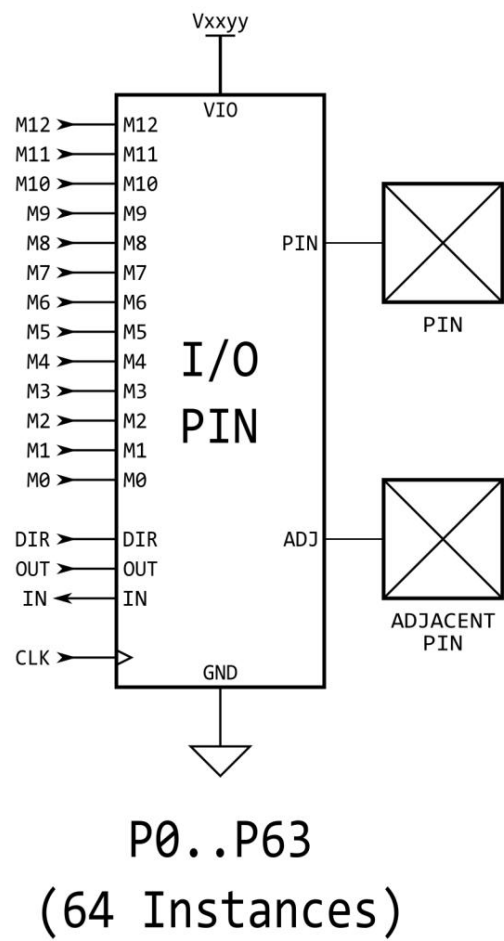
¹ 用于反馈操作;提供连续 (非时钟)信号

² 信号仅在时钟边沿更新

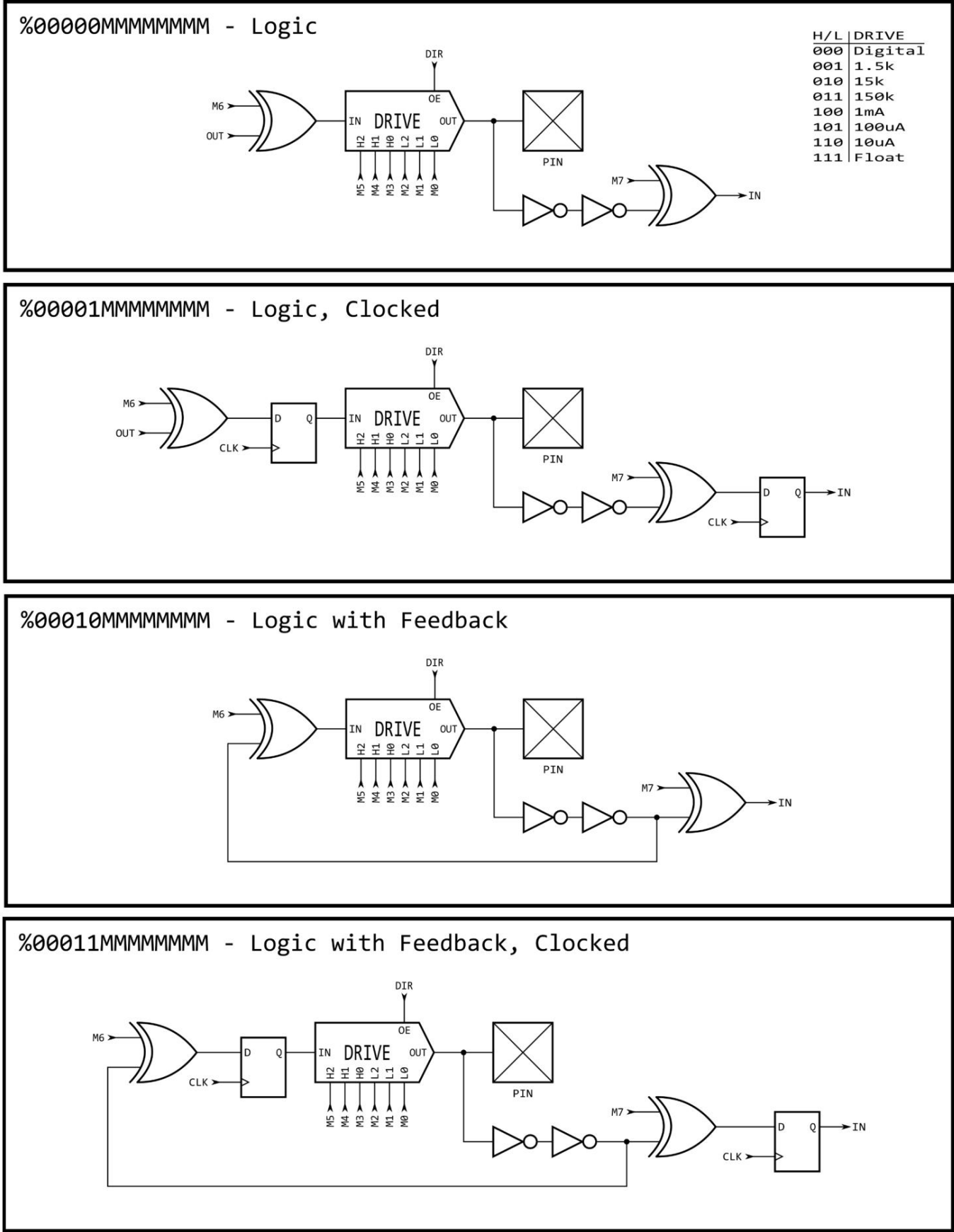
(T)引脚 DIR/OUT 控制		
默认值 (%TT = 00)		
对于奇数引脚	OTHER = 偶数引脚的非（反转）输出状态（差分源）	
对于偶数针	“其他”= 唯一伪随机位（噪声源）	
对于所有引脚	SMART = 智能引脚输出,覆盖 OUT/OTHER	
当 M[12:10] = %101 时启用 ‘DAC_MODE’		
在 DAC_MODE 中, BIT_DAC 输出 {2{M[7:4]}} 表示 高 或 {2{M[3:0]}} 表示 低		
智能密码模式 “关闭” (%SSSSS = %00000)		
	DIR 使能输出	
	对于非DAC_MODE	
	0x	OUT驱动输出
	1x	其他驱动输出
	对于 DAC_MODE	
	00	DIR启用DAC,M[7:0]设置DAC电平
	01	OUT 使能 ADC,M[3:0] 选择 cog DAC 通道
	10	OUT驱动BIT_DAC
	11	其他驱动器 BIT_DAC
智能密码模式 “开启” (%SSSSS > %00000)		
	x0	输出禁用,无论 DIR 如何
	x1	输出使能,无论 DIR 如何
用于 DAC 智能引脚模式 (%SSSSS = %00001..%00011)		
	0x	OUT 在 DAC_MODE 下启用 DAC,M[7:0] 被覆盖
	1x	OTHER 在 DAC_MODE 中启用 DAC,M[7:0] 被覆盖
对于非 DAC 智能引脚模式 (%SSSSS = %00100..%11111)		
	0x	SMART/OUT 驱动输出,或如果 DAC_MODE 则驱动 BIT_DAC
	1x	SMART/OTHER 驱动输出,或如果 DAC_MODE 则为 BIT_DAC

I/O 引脚电路下图为

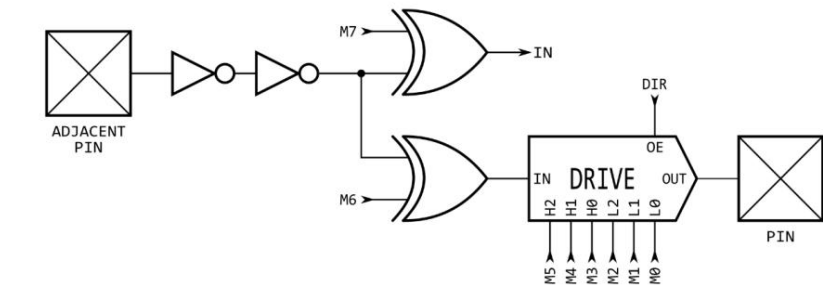
单个 I/O 引脚电路图,该引脚由其本地 3.3V 电源引脚 (Vxxyy) 供电。它连接到其自身的物理引脚 (PIN),以及相邻的奇数或偶数引脚 (ADJ)。
I/O 引脚 P0 和 P1 彼此视为相邻引脚,P2 和 P3 也是如此,等等。



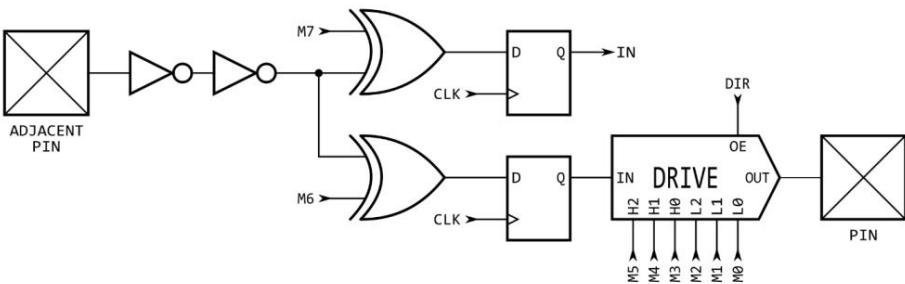
每个独特 I/O 引脚配置的等效原理图



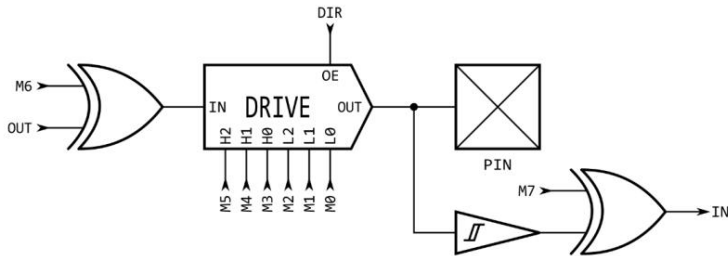
%00100MMMMMMMM - Logic with Adjacent-Pin Feedback



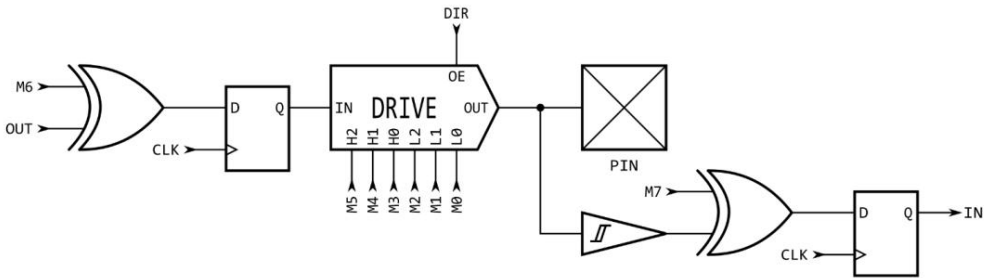
%00101MMMMMMMM - Logic with Adjacent-Pin Feedback, Clocked



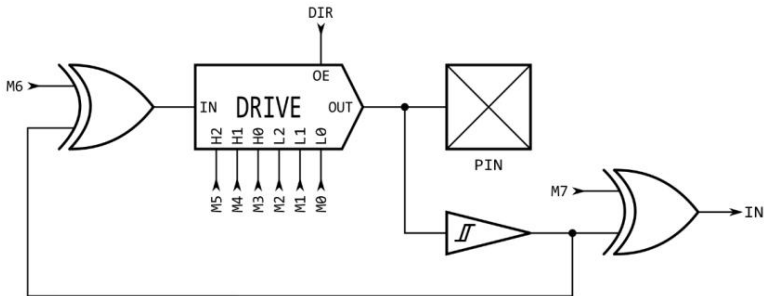
%00110MMMMMMMM - Schmitt



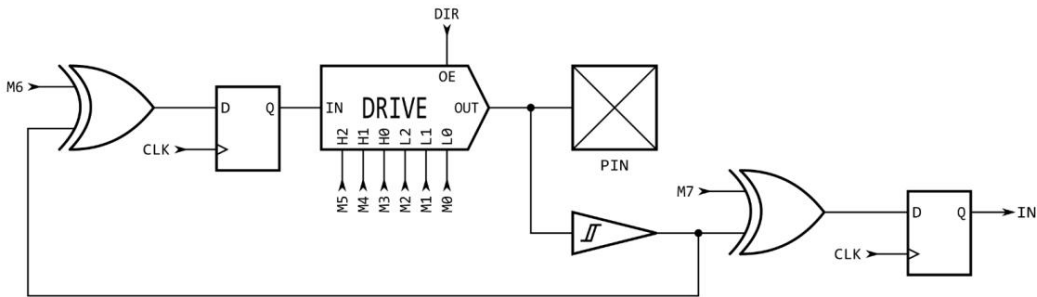
%00111MMMMMMMM - Schmitt, Clocked



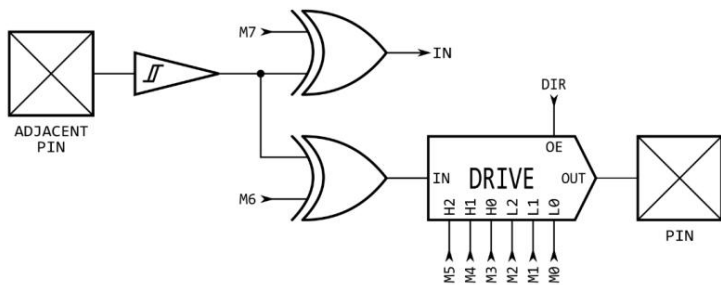
%0100MMMMMMMM - Schmitt with Feedback



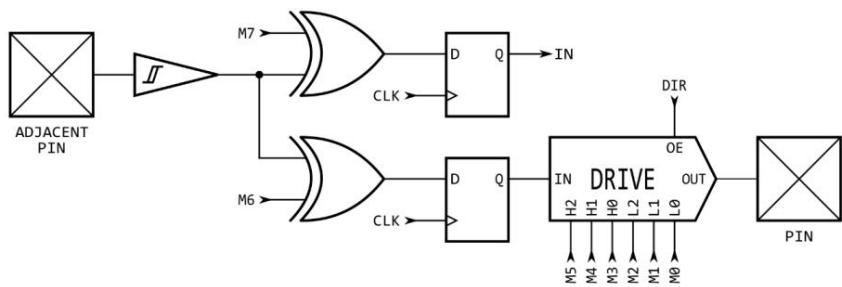
%01001MMMMMMMM - Schmitt with Feedback, Clocked



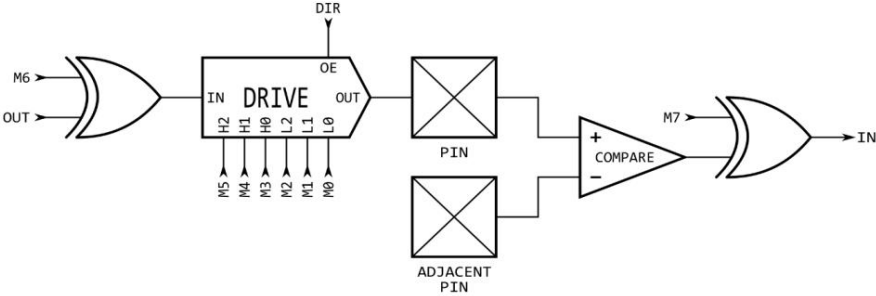
%01010MMMMMMMM - Schmitt with Adjacent-Pin Feedback



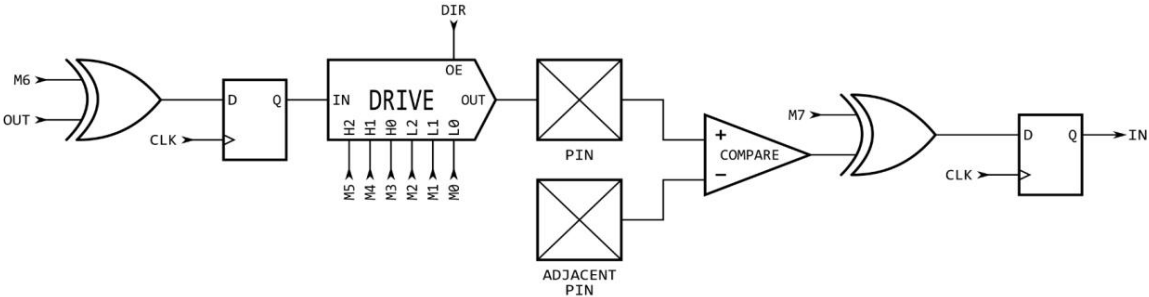
%01011MMMMMMMM - Schmitt with Adjacent-Pin Feedback, Clocked



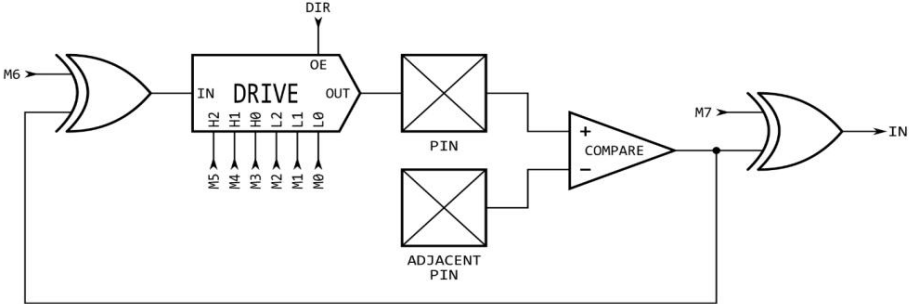
%01100MMMMMMMM - Comparator



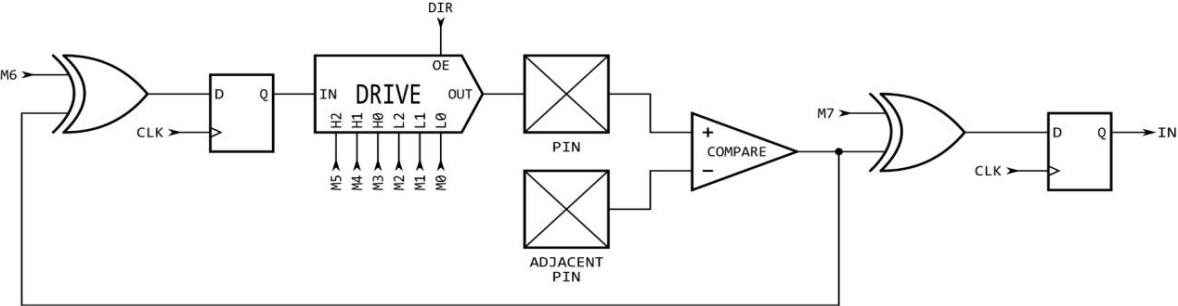
%01101MMMMMMMM - Comparator, Clocked



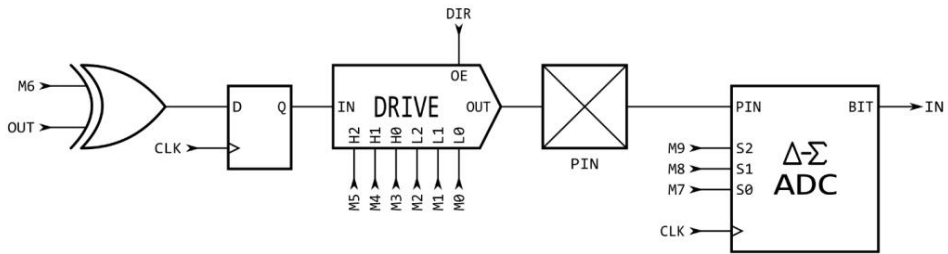
%01110MMMMMMMM - Comparator with Feedback



%01111MMMMMMMM - Comparator with Feedback, Clocked

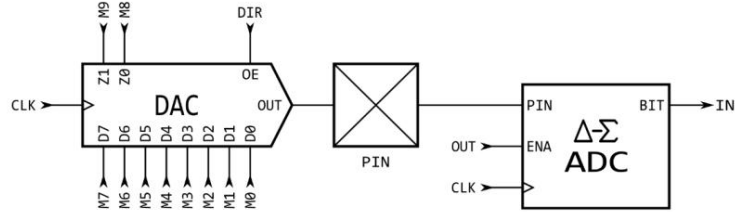


%100MMMMMMMMMM - ADC with Optional Drive



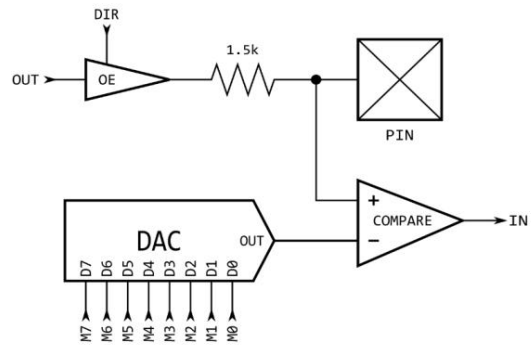
SSS	ADC
000	GND
001	VIO
010	Float
011	1x
100	3.2x
101	10x
110	32x
111	100x

%101MMMMMMMMMM - DAC with Optional ADC

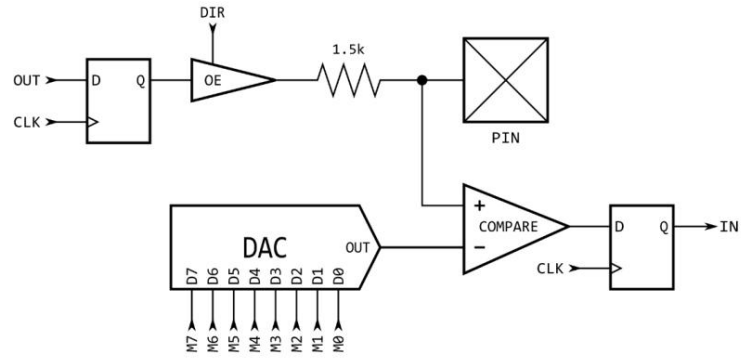


ZZ	DAC
00	990 ohm 3.3V
01	600 ohm 2.0V
10	124 ohm 3.3V
11	75 ohm 2.0V

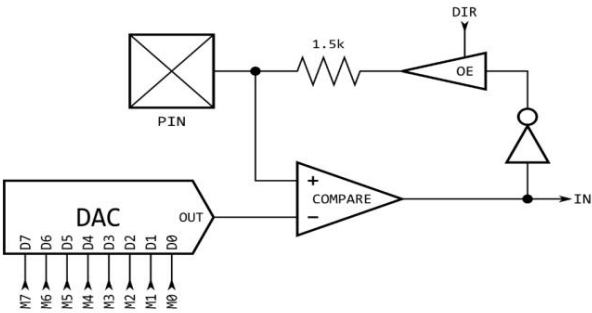
%11000MMMMMMMM - Level Comparator with 1.5k Output



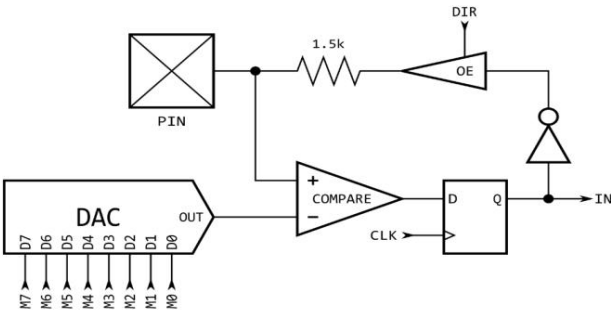
%11001MMMMMMMM - Level Comparator with 1.5k Output, Clocked



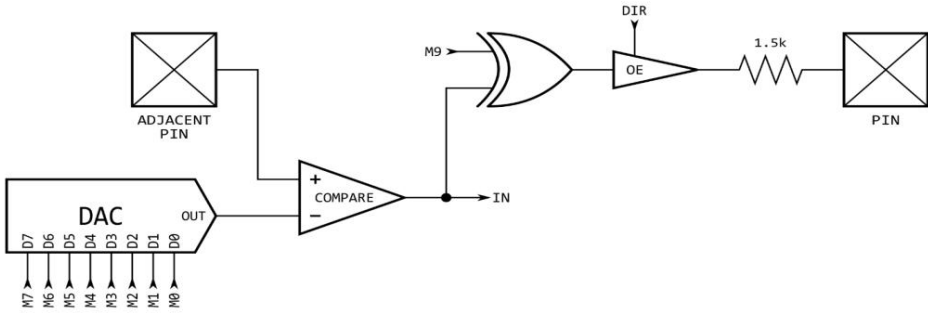
%11010MMMMMMMM - Level Comparator with Local Feedback



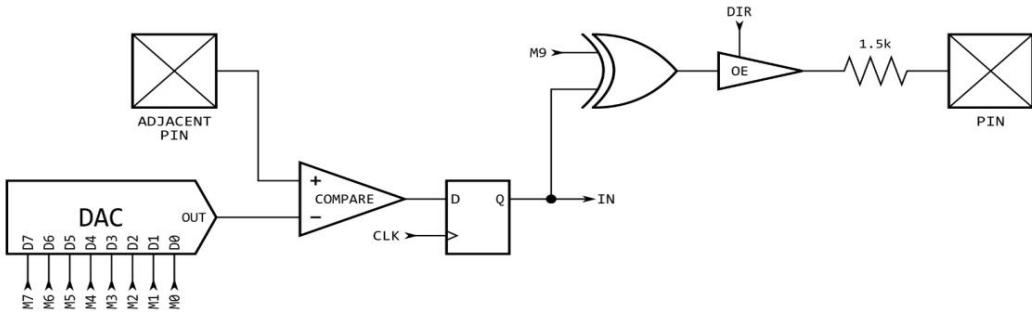
%11011MMMMMMMM - Level Comparator with Local Feedback, Clocked



%111M0MMMMMMMM - Level Comparator with Separate Feedback



%111M1MMMMMMMM - Level Comparator with Separate Feedback, Clocked



智能模式

每个 I/O 引脚都有内置的“智能引脚”电路,当启用时,它可以执行 34 种不同的自主操作之一
引脚上的功能。智能引脚通过提供
高带宽并发硬件功能,否则 cogs 无法通过 I/O 引脚执行
操纵指令。

在正常运行中,I/O 引脚的输出使能由其 DIR 位控制,其输出状态由其 OUT 位控制
位,其 IN 位返回引脚的读取状态。启用智能引脚模式后,其 DIR 位用作低电平有效
复位信号发送到智能引脚电路,而输出使能状态由配置位控制。在某些情况下
模式时,智能引脚电路将接管驱动输出状态,在这种情况下,OUT 位将被忽略。其 IN 位
作为一个标志,向齿轮指示智能引脚已完成某项功能或事件已发生
发生了,也许需要承认。

要配置智能引脚,首先将其 DIR 位设置为低(保持复位状态),然后使用 WRPIN、WXPIN 和 WYPIN 来
建立模式和相关参数。配置完成后,DIR 可以升高,智能引脚将开始
操作。之后,根据模式,您可以通过 WXPIN/WYPIN 向其输入新数据,或使用以下方式检索结果
RDPIN/RQPIN。这些活动通常与 IN 信号升高相协调;稍后解释。

请注意,在配置智能引脚时,%TT 位 (WRPIN 指令的 D 操作数)将控制引脚的
输出使能,无论 DIR 状态如何。

智能引脚内部有四个 32 位寄存器:

智能引脚寄存器	
32位寄存器用途	
模式	智能引脚模式,以及低级 I/O 引脚模式 (只写)
+	模式特定参数 (只写)
是	模式特定参数 (只写)
Z	模式特定结果 (只读)

这四个寄存器通过以下 2 个时钟指令进行写入和读取,其中 S/# 用于选择
引脚号 (0..63)和 D/# 是 32 位数据管道:

WRPIN D/#,S/#	- 将智能密码 S/# 模式设置为 D/#,确认密码
WXPIN D/#,S/#	- 将智能引脚 S/# 参数 X 设置为 D/#,确认引脚
WYPIN D/#,S/#	- 将智能引脚 S/# 参数 Y 设置为 D/#,确认引脚
RDPIN D,S/# {WC}	- 将智能引脚 S/# 结果 Z 放入 D,将标志放入 C,确认引脚
RQPIN D,S/# {WC}	- 将智能密码 S/# 结果 Z 放入 D,将标志放入 C,不确认密码
AKPIN S/#	- 确认引脚S/#

每个智能引脚都有一个 34 位输入总线和一个 33 位输出总线,将其连接到齿轮。

为了配置和控制智能引脚,每个齿轮将数据和确认信号写入智能引脚输入
总线。每个智能引脚以相同的方式对来自 cogs 集合的所有传入 34 位总线进行“或”操作,DIR 和 OUT 位
在到达引脚之前进行“或”运算。因此,如果您打算使用多个齿轮,请执行 WRPIN / WXPIN / WYPIN /
RDPIN / AKPIN 指令在同一智能密码上执行,您必须确保它们在不同时间执行,以便
避免破坏彼此的总线数据。使用 RDPIN 读取智能引脚可能会导致同样的冲突;但是,

通过使用 RQPIN（“读取安静”），任意数量的 cog 都可以同时读取智能引脚，而不会发生总线冲突，因为它不利用智能引脚输入总线来发送确认信号（就像 RDPIN 那样）。

每个智能引脚都会写入其输出总线，以传达其 Z 结果和一个特殊标志。RDPIN 和 RQPIN 复用并读取这些总线，以便将引脚的 Z 结果读入 D，并将其特殊标志读入 C。C 可以是模式相关标志或 Z 结果的 MSB。

当智能引脚上发生与模式相关的事件时，它会发出 IN 信号，提醒齿轮新数据已准备就绪，可以加载新数据，或者某个进程已完成。cog 可以通过 TESTP 指令测试此信号并可以通过执行 WRPIN、WXPIN、WYPIN、RDPIN 或 AKPIN 指令来确认智能密码。这确认会导致智能引脚降低其 IN 信号，以便在下一个事件发生时可以再次升高。在 WRPIN / WXPIN / WYPIN / RDPIN / AKPIN 之后，需要两个时钟才能使 IN 下降，然后才能再次进行轮询。

智能引脚可以随时重置，无需重新配置，只需清除然后设置其 DIR 位即可。

要将密码恢复到正常模式，请执行“WRPIN #0,pin”。

(S)智能密码模式		
%SSSSS 模式		笔记
00000	智能引脚关闭;正常运行（默认）	
00001	长存储库	M[12:10] != %101（非 DAC_MODE）
00010	长存储库	M[12:10] != %101（非 DAC_MODE）
00011	长存储库	M[12:10] != %101（非 DAC_MODE）
00001 DAC噪声		M[12:10] = %101 (DAC_模式)
00010 DAC 16位抖动、噪声		M[12:10] = %101 (DAC_模式)
00011 DAC 16位抖动,PWM		M[12:10] = %101 (DAC_模式)
001001脉冲/周期输出		
001011转换输出		
001101 NCO频率		
001111士官职责		
010001 PWM三角波		
010011 PWM锯齿波		
010101 PWM开关电源,V和反馈		
01011	周期/连续:AB正交编码器	
01100	周期性/连续性:在 A 上升和 B 高时增加	
01101	周期性/连续性:在 A 上升和 B 高时增加 / 在 A 上升和 B 低时减少	
01110	周期性/连续性:A 点上升时增加,B 点上升时减少}	
01111	周期/连续:在 A 高时增加 { 在 B 高时减少}	
10000 次 A 状态		
10001	时间 A 高	
10010 时间 X A-高点/上升/边沿 - 或 - X A-高点/上升/边沿超时		
10011	对于 X 个周期,计算时间	

10100 表示 X 个周期,计数状态		
10101 对于 X+ 个时钟周期,计数时间		
10110 表示 X+ 个时钟周期,计数状态		
10111 对于 X+ 个时钟周期,计数周期		
11000 ADC 采样/过滤/捕获,内部时钟		
11001 ADC 采样/滤波/捕获,外部时钟		
带触发器的 11010 ADC 示波器		
110111 USB 主机/设备		偶数/奇数引脚对 = DM/DP
111001 同步串行传输		A 数据,B 时钟
11101 同步串行接收		A 数据,B 时钟
111101 异步串行传输		波特率
11111 异步串行接收		波特率

¹ OUT 信号被覆盖

重启

在正常供电的情况下,如果 Propeller 2 在 RESn 引脚上收到低脉冲或执行 HUBSET,则它会重新启动 ##\$1000_0000 指令。两种复位方法,外部复位 (通过 RESn 引脚)和内部复位 (通过 HUBSET) ,都表现为相同;但是,无法使用 RESn 引脚从外部检测内部复位。

PROPELLER 2 汇编语言 (PASM2) 简介

数学和逻辑指令		
操作说明	描述	时钟 Reg,LUT 和 Hub
ABS D {WC/WZ/WCZ}	将 D 的绝对值代入 D。D = ABS(D)。C = D[31]。*	2
ABS D,{#}S {WC/WZ/WCZ}	将 S 的绝对值代入 D。D = ABS(S)。C = S[31]。*	2
添加 D,{#}S {WC/WZ/WCZ}	将 S 添加到 D 中。D = D + S。C = (D + S) 的进位。*	2
添加 D,{#}S {WC/WZ/WCZ}	将 S 加到 D 中,带符号。D = D + S。C = (D + S) 的正确符号。*	2
ADDSX D,{#}S {WC/WZ/WCZ}	将 (S + C) 加到 D 中,进行符号扩展。D = D + S + C。C = (D + S + C) 的正确符号。Z = Z AND (结果 == 0)。	2
ADDX D,{#}S {WC/WZ/WCZ}	将 (S + C) 添加到 D 中,扩展。D = D + S + C。C = (D + S + C) 的进位。Z = Z AND (结果 == 0)。	2
和 D,{#}S {WC/WZ/WCZ}	AND S 代入 D。D = D & S。C = 结果的奇偶校验。*	2
安德 D,{#}S {WC/WZ/WCZ}	AND IS 代入 D。D = D & IS。C = 结果的奇偶校验。*	2
BITC D,{#}S {WCZ}	位 D[S[9:5]+S[4:0]:S[4:0]] = C。其他位不受影响。之前的 SETQ 会覆盖 S[9:5]。C,Z = 原始 D[S[4:0]]。	2
BITH D,{#}S {WCZ}	位 D[S[9:5]+S[4:0]:S[4:0]] = 1。其他位不受影响。之前的 SETQ 覆盖 S[9:5]。C,Z = 原始 D[S[4:0]]。	2
BITL D,{#}S {WCZ}	位 D[S[9:5]+S[4:0]:S[4:0]] = 0。其他位不受影响。之前的 SETQ 会覆盖 S[9:5]。C,Z = 原始 D[S[4:0]]。	2
BITNC D,{#}S {WCZ}	位 D[S[9:5]+S[4:0]:S[4:0]] = !C。其他位不受影响。之前的 SETQ 会覆盖 S[9:5]。C,Z = 原始 D[S[4:0]]。	2
比特诺特 D,{#}S {WCZ}	切换位 D[S[9:5]+S[4:0]:S[4:0]]。其他位不受影响。之前的 SETQ 会覆盖 S[9:5]。C,Z = 原始 D[S[4:0]]。	2
BITNZ D,{#}S {WCZ}	位 D[S[9:5]+S[4:0]:S[4:0]] = !Z。其他位不受影响。之前的 SETQ 会覆盖 S[9:5]。C,Z = 原始 D[S[4:0]]。	2
BITRND D,{#}S {WCZ}	位 D[S[9:5]+S[4:0]:S[4:0]] = RND。其他位不受影响。之前的 SETQ 会覆盖 S[9:5]。C,Z = 原始 D[S[4:0]]。	2
比茨 D,{#}S {WCZ}	位 D[S[9:5]+S[4:0]:S[4:0]] = Z。其他位不受影响。之前的 SETQ 会覆盖 S[9:5]。C,Z = 原始 D[S[4:0]]。	2
屏蔽	将大小为 (D[4:0] + 1) 的 LSB 对齐位掩码放入 D 中。D = (\$0000_0002 << D[4:0]) - 1。	2
BMASK D,{#}S	将大小为 (S[4:0] + 1) 的 LSB 对齐位掩码放入 D 中。D = (\$0000_0002 << S[4:0]) - 1。	2
比较 D,{#}S {WC/WZ/WCZ}	比较 D 与 S。C = (D - S) 的借位。Z = (D == S)。	2
CMPM D,{#}S {WC/WZ/WCZ}	比较 D 与 S,将差值的 MSB 代入 C。C = (D - S) 的 MSB。Z = (D == S)。	2
CMPR D,{#}S {WC/WZ/WCZ}	比较 S 与 D (反向)。C = (S - D) 的借位。Z = (D == S)。	2
CMPS D,{#}S {WC/WZ/WCZ}	比较 D 与 S,带符号。C = (D - S) 的正确符号。Z = (D == S)。	2
CMPSUB D,{#}S {WC/WZ/WCZ}	如果 D >= S,则比较并从未 D 中减去 S。如果 D < S 则 D = D - S 且 C = 1,否则 D 相同且 C = 0。*	2
CMPSX D,{#}S {WC/WZ/WCZ}	比较 D 与 (S + C),进行符号扩展。C = (D - (S + C)) 的正确符号。Z = Z 且 (D == S + C)。	2
康普赛 D,{#}S {WC/WZ/WCZ}	比较 D 与 (S + C) 扩展。C = (D - (S + C)) 的借位。Z = Z AND (D == S + C)。	2
CRCBIT D,{#}S	使用 C 和 S 中的多项式迭代 D 中的 CRC 值。如果 (C XOR D[0]) 则 D = (D >> 1) XOR S,否则 D = (D >> 1)。	2
CRCNIB D,{#}S	使用 Q[31:28] 和 S 中的多项式迭代 D 中的 CRC 值。例如 CRCBIT x 4。Q = Q << 4。使用 REP #n,#1 +SETQ+CRCNIB+CRCNIB+CRCNIB...	2
DECMOD D,{#}S {WC/WZ/WCZ}	模数递减。如果 D = 0,则 D = S 且 C = 1,否则 D = D - 1 且 C = 0。*	2
解码	将 D[4:0] 解码为 D。D = 1 << D[4:0]。	2
解码 D,{#}S	将 S[4:0] 解码为 D。D = 1 << S[4:0]。	2
编码 {WC/WZ/WCZ}	将 D 中最高位 '1' 的位置放入 D 中。D = S 中最高位 '1' 的位置(0..31)。C = (S != 0)。*	2
ENCOD D,{#}S {WC/WZ/WCZ}	将 S 中最高位 '1' 的位置存入 D。D = S 中最高位 '1' 的位置 (0..31)。C = (S != 0)。*	2
费氏数 D,{#}S {WC/WZ/WCZ}	强制 D >= S。如果 D < S 则 D = S 且 C = 1,否则 D 相同且 C = 0。*	2
费格斯 D,{#}S {WC/WZ/WCZ}	强制 D >= S,带符号。如果 D < S,则 D = S 且 C = 1,否则 D 相同,C = 0。*	2
弗莱明 D,{#}S {WC/WZ/WCZ}	强制 D <= S。如果 D > S 则 D = S 且 C = 1,否则 D 相同且 C = 0。*	2
弗莱斯 D,{#}S {WC/WZ/WCZ}	强制 D <= S,带符号。如果 D > S,则 D = S 且 C = 1,否则 D 相同,C = 0。*	2
获取字节 D	将先前的 ALTGB 指令建立的字节放入 D 中。	2
获取字节 D,{#}S,#N	将 S 的字节 N 放入 D。D = {24 b0, S.BYTE[N]}。	2
GETNIB D	将由先前的 ALTGN 指令建立的半字节放入 D 中。	2
GETNIB D,{#}S,#N	将 S 的半字节 N 放入 D。D = {28 b0, S.NIBBLE[N]}。	2
GETWORD D	将先前的 ALTGW 指令建立的字放入 D 中。	2
获取单词 D,{#}S,#N	将 S 的单词 N 放入 D。D = {16 b0, S.WORD[N]}。	2
INCMOD D,{#}S {WC/WZ/WCZ}	模数递增。如果 D = S 则 D = 0 且 C = 1,否则 D = D + 1 且 C = 0。*	2
位置 PA/PB/PTRA/PTRB,{#}\A	将 {12 b0, address[19:0]} 放入 PA/PB/PTRA/PTRB (每个 W)。如果 R = 1,则地址 = PC + A,否则地址 = A。 \ 力 R = 0。	2

合并D		合并 D 中的字节位。D = {D[31], D[23], D[15], D[7], ...D[24], D[16], D[8], D[0]}。	2
合并D		合并 D 中字的位。D = {D[31], D[15], D[30], D[14], ...D[17], D[1], D[16], D[0]}。	2
国防部	c	{WC}根据 cccc 修改 C。C = cccc[{C,Z}]。	2
MODCZc,z	{WC/WZ/WCZ}	根据 cccc 和 zzzz 修改 C 和 Z。C = cccc[{C,Z}], Z = zzzz[{C,Z}]。	2
MODZ	z	{WZ}根据 zzzz 修改 Z。Z = zzzz[{C,Z}]。	2
移动	D _i {#}S {WC/WZ/WCZ}	将 S 移入 D。D = S。C = S[31]。	2
MOVBYTS	D _i {#}S	按 S 在 D 内移动字节。D = {D.BYTE[S[7:6]], D.BYTE[S[5:4]], D.BYTE[S[3:2]], D.BYTE[S[1:0]]}。	2
MUL	D _i {#}S	{WZ} D = 无符号 (D[15:0] * S[15:0])。Z = (S == 0) (D == 0)。	2
穆尔	D _i {#}S	{WZ} D = 有符号 (D[15:0] * S[15:0])。Z = (S == 0) (D == 0)。	2
多路复用器	D _i {#}S {WC/WZ/WCZ}	将 C 多路复用到 S 中每个为 1 的 D 位中。D = (S & D) (S & {32{C}})。C = 结果的奇偶校验。	2
MUXNC	D _i {#}S {WC/WZ/WCZ}	将 C 多路复用到 S 中每个为 1 的 D 位中。D = (S & D) (S & {32{C}})。C = 结果的奇偶校验。	2
MUXNIBS	D _i {#}S	对于 S 中的每个非零半字节,将其复制到相应的 D 半字节中,否则将该 D 半字节保留相同的。	2
MUXNITS	D _i {#}S	对于 S 中的每个非零位,将该位复制到相应的 D 位中,否则将该 D 位对保留为相同的。	2
MUXNZ	D _i {#}S {WC/WZ/WCZ}	将 IZ 多路复用到 S 中每个为 1 的 D 位中。D = (S & D) (S & {32{IZ}})。C = 结果的奇偶校验。	2
多路复用器	D _i {#}S	在 SETQ 之后使用。对于 Q 中的每个 “1”位,将 S 中对应的位复制到 D 中。D = (D & Q) (S & Q)。	2
多路复用器	D _i {#}S {WC/WZ/WCZ}	将 Z 多路复用到 S 中每个为 1 的 D 位中。D = (S & D) (S & {32{Z}})。C = 结果的奇偶校验。	2
阴性	D	{WC/WZ/WCZ}对 D 取反。D = -D。C = 结果的 MSB。	2
阴性	D _i {#}S {WC/WZ/WCZ}	将 S 取反并代入 D。D = -S。C = 结果的 MSB。	2
东北风天气公司	D	{WC/WZ/WCZ}用 C 对 D 取反。如果 C = 1 则 D = -D,否则 D = D。C = 结果的 MSB。	2
东北风天气公司	D _i {#}S {WC/WZ/WCZ}	将 S 除以 C 并代入 D。如果 C = 1 则 D = -S,否则 D = S。C = 结果的 MSB。	2
负面		{WC/WZ/WCZ}用 IC 对 D 取反。如果 C = 0 则 D = -D,否则 D = D。C = 结果的 MSB。	2
NEGNC	D _i {#}S {WC/WZ/WCZ}	通过 IC 对 S 取反并放入 D 中。如果 C = 0 则 D = -S,否则 D = S。C = 结果的 MSB。	2
NEGNZ	D	{WC/WZ/WCZ}用 IZ 对 D 取反。如果 Z = 0 则 D = -D,否则 D = D。C = 结果的 MSB。	2
NEGNZ	D _i {#}S {WC/WZ/WCZ}	将 S 用 IZ 取反并放入 D 中。如果 Z = 0 则 D = -S,否则 D = S。C = 结果的 MSB。	2
阴性	D	{WC/WZ/WCZ}用 Z 对 D 取反。如果 Z = 1 则 D = -D,否则 D = D。C = 结果的 MSB。	2
阴性	D _i {#}S {WC/WZ/WCZ}	将 S 除以 Z 并计算 D 的值。如果 Z = 1 则 D = -S,否则 D = S。C = 结果的 MSB。	2
不是	D	{WC/WZ/WCZ}将 ID 代入 D。D = !D。C = !D[31]。	2
不是	D _i {#}S {WC/WZ/WCZ}	将 IS 代入 D。D = !S。C = !S[31]。	2
一	D	{WC/WZ/WCZ}将 D 中的 “1”的个数放入 D。D = S 中 “1”的个数 (0..32)。C = 结果的最低有效位 (LSB)。	2
一	D _i {#}S {WC/WZ/WCZ}	将 S 中的 “1”的个数放入 D 中。D = S 中 “1”的个数 (0..32)。C = 结果的最低有效位 (LSB)。	2
或者	D _i {#}S {WC/WZ/WCZ}	或 S 到 D。D = D S。C = 结果的奇偶校验。	2
旋转移位	D _i {#}S {WC/WZ/WCZ}	循环左移进位。D = ({D[31:0], {32{C}}} << S[4:0]) 中的 [63:32]。如果 S[4:0] > 0,则 C = 移出最后一位,否则为 D[31]。	2
旋转移位	D _i {#}S {WC/WZ/WCZ}	循环右移进位。D = ({({32{C}}, D[31:0]) >> S[4:0]) 中的 [31:0]。如果 S[4:0] > 0,则 C = 最后一位移出,否则为 D[0]。	2
RCZL	D	{WC/WZ/WCZ}将 C,Z 循环左移至 D。D = {D[29:0], C, Z}。C = D[31], Z = D[30]。	2
RCZR	D	{WC/WZ/WCZ}将 C,Z 循环右移至 D。D = {C, Z, D[31:2]}。C = D[1], Z = D[0]。	2
修订版	D	反转 D 位。D = D[0:31]。	2
RGBEXP	D	将 D[15:0] 中的 5:6:5 RGB 值扩展为 D[31:8] 中的 8:8:8 值。D = {D[15:11,15:13], D[10:5,10:9], D[4:0,4:2], 8 b0}。	2
RGBSQZ	D	将 D[31:8] 中的 8:8:8 RGB 值压缩为 D[15:0] 中的 5:6:5 值。D = {15 b0, D[31:27], D[23:18], D[15:11]}。	2
角色扮演	D _i {#}S {WC/WZ/WCZ}	左移。D = ({D[31:0], D[31:0]} << S[4:0]) 中的 [63:32]。如果 S[4:0] > 0,则 C = 最后一位移出,否则为 D[31]。	2
ROLBYTE	D	将前一个 ALTGB 指令建立的字节左移到 D 中。	2
ROLBYTE	D _i {#}S,#N	将 S 的字节 N 左移到 D。D = {D[23:0], S.BYTE[N]}。	2
罗尔尼布		将由先前的 ALTGN 指令建立的半字节左移到 D 中。	2
ROLNIB	D _i {#}S,#N	将 S 的半字节 N 左循环至 D。D = {D[27:0], S.NIBBLE[N]}。	2
罗尔沃德		将由先前的 ALTGW 指令建立的字循环左移到 D 中。	2
ROLWORD	D _i {#}S,#N	将 S 的字 N 左移到 D。D = {D[15:0], S.WORD[N]}。	2
回报率	D _i {#}S {WC/WZ/WCZ}	右移。D = ({D[31:0], D[31:0]} >> S[4:0]) 中的 [31:0]。如果 S[4:0] > 0,则 C = 最后一位移出,否则为 D[0]。	2
萨尔	D _i {#}S {WC/WZ/WCZ}	左移算术。D = ({D[31:0], {32{D[0]}} << S[4:0]) 的 [63:32]。如果 S[4:0] > 0,则 C = 最后一位移出,否则为 D[31]。	2
特区	D _i {#}S {WC/WZ/WCZ}	右移算术运算。D = ({({32{D[31]}}, D[31:0]) >> S[4:0]) 的 [31:0]。如果 S[4:0] > 0,则 C = 最后一位移出,否则为 D[0]。	2

声门控	D _i {#}S	{WZ}下一条指令的 S 值 = 无符号 (D[15:0] * S[15:0]) >> 16. * {WZ}下一条指令的	2
指令寄存器	D _i {#}S	S 值 = 有符号 (D[15:0] * S[15:0]) >> 14. 在此方案中, \$4000 = 1.0 且 \$C000 = -1.0. *	2
SETBYTE {#}S		将 S[7:0] 设置为由之前的 ALTSB 指令建立的字节。	2
设置字节 D _i {#}S, #N		将 S[7:0] 设置为 D 中的字节 N,保持 D 的其余部分不变。	2
SETD D _i {#}S		将D的D字段设置为S[8:0]。D = {D[31:18], S[8:0], D[8:0]}。	2
SETNIB {#}S		将 S[3:0] 设置为由之前的 ALTSN 指令建立的半字节。	2
SETNIB D _i {#}S, #N		将 S[3:0] 设置为 D 中的半字节 N,保持 D 的其余部分不变。	2
塞特	D _i {#}S	将D的R字段设置为S[8:0]。D = {D[31:28], S[8:0], D[18:0]}。	2
套	D _i {#}S	将D的S字段设置为S[8:0]。D = {D[31:9], S[8:0]}。	2
设置字 {#}S		将 S[15:0] 设置为由之前的 ALTSW 指令建立的字。	2
设置字 D _i {#}S, #N		将 S[15:0] 设置为 D 中的字 N,保持 D 的其余部分不变。	2
苏斯博士		重新定位并定期反转 D 中的位。在第 32 次迭代时返回原始值。正向模式。	2
苏斯博士		重新定位并定期反转 D 中的位。在第 32 次迭代时返回原始值。反转模式。	2
SHL D _i {#}S {WC/WZ/WCZ}	左移。D = ((D[31:0], 32 - b0) << S[4:0]) 中的 [63:32]。如果 S[4:0] > 0,则 C = 最后一位移出,否则为 D[31]。*		2
移位寄存器	D _i {#}S {WC/WZ/WCZ}	右移。D = ((32 - b0, D[31:0]) >> S[4:0]) 中的 [31:0]。如果 S[4:0] > 0,则 C = 最后一位移出,否则为 D[0]。*	2
SIGNX D _i {#}S {WC/WZ/WCZ}	从位 S[4:0] 对 D 进行符号扩展。C = 结果的 MSB。*		2
分裂D		将 D 的每 4 位拆分为字节。D = {D[31], D[27], D[23], D[19], ...D[12], D[8], D[4], D[0]}。	2
分裂D		将 D 的奇数/偶数位拆分为字。D = {D[31], D[29], D[27], D[25], ...D[6], D[4], D[2], D[0]}。	2
子	D _i {#}S {WC/WZ/WCZ}	从 D 中减去 S。D = D - S。C = (D - S) 的借位。*	2
SUBR D _i {#}S {WC/WZ/WCZ}	从 S 中减去 D (反向)。D = S - D。C = (S - D) 的借位。*		2
替补	D _i {#}S {WC/WZ/WCZ}	从 D 中减去 S,带符号。D = D - S。C = (D - S) 的正确符号。*	2
SUBSX D _i {#}S {WC/WZ/WCZ}		从 D 中减去 (S + C),进行符号扩展。D = D - (S + C)。C = (D - (S + C)) 的正确符号。Z = Z AND (结果 == 0)。	2
子目录	D _i {#}S {WC/WZ/WCZ}	从 D 中减去 (S + C),扩展。D = D - (S + C)。C = (D - (S + C)) 的借位。Z = Z AND (结果 == 0)。	2
SUMC D _i {#}S {WC/WZ/WCZ}	将 +/-S 除以 C 并入 D。如果 C = 1 则 D = D - S,否则 D = D + S。C = (D +/- S) 的正确符号。*		2
SUMNC D _i {#}S {WC/WZ/WCZ}	使用 !C 将 +/-S 加到 D 中。如果 C = 0,则 D = D - S,否则 D = D + S。C = (D +/- S) 的正确符号。*		2
SUMNZ D _i {#}S {WC/WZ/WCZ}	用 !Z 将 +/-S 加到 D 中。如果 Z = 0,则 D = D - S,否则 D = D + S。C = (D +/- S) 的正确符号。*		2
萨姆兹	D _i {#}S {WC/WZ/WCZ}	将 +/-S 除以 Z 并入 D。如果 Z = 1 则 D = D - S,否则 D = D + S。C = (D +/- S) 的正确符号。*	2
测试 D	{WC/WZ/WCZ}	测试 D。C = D 的奇偶校验。Z = (D == 0)。	2
测试 D _i {#}S {WC/WZ/WCZ}	用 S 检验 D。C = (D & S) 的奇偶校验。Z = ((D & S) == 0)。		2
测试B D _i {#}S	WC/WZ	测试 D 的位 S[4:0],写入 C/Z。C/Z = D[S[4:0]]。	2
测试B D _i {#}S	ORC/ORZ	测试 D 的位 S[4:0],或入 C/Z。C/Z = C/Z 或 D[S[4:0]]。	2
测试B D _i {#}S	ANDC/ANDZ	测试 D 的位 S[4:0],并将其与 C/Z 进行与运算。C/Z = C/Z AND D[S[4:0]]。	2
测试B D _i {#}S	XORC/XORZ	测试 D 的位 S[4:0],将其异或到 C/Z。C/Z = C/Z XOR D[S[4:0]]。	2
测试BN D _i {#}S	WC/WZ	测试 D 的位 S[4:0],写入 C/Z。C/Z = !D[S[4:0]]。	2
测试BN D _i {#}S	ORC/ORZ	测试 D 的位 S[4:0],或进入 C/Z。C/Z = C/Z 或 !D[S[4:0]]。	2
测试BN D _i {#}S	ANDC/ANDZ	测试 D 的位 S[4:0],并将其与 C/Z 进行与运算。C/Z = C/Z AND !D[S[4:0]]。	2
测试BN D _i {#}S	XORC/XORZ	测试 D 的位 S[4:0],将其异或到 C/Z。C/Z = C/Z XOR !D[S[4:0]]。	2
测试D _i {#}S	{WC/WZ/WCZ}	用 !S 测试 D。C = (D & !S) 的奇偶校验。Z = ((D & !S) == 0)。	2
指令寄存器	D	根据 C,将 0 或 1 写入 D。D = {31 - b0, C}。	2
指令寄存器	D	根据 !C,将0或1写入D。D = {31 - b0, !C}。	2
指令寄存器	D	根据 !Z,将0或1写入D。D = {31 - b0, !Z}。	2
WRZ D		根据 Z,将 0 或 1 写入 D。D = {31 - b0, Z}。	2
异或 D _i {#}S {WC/WZ/WCZ}	将 S 与 D 进行异或。D = D ^ S。C = 结果的奇偶校验。*		2
XORO32 D		使用 xoroshiro32+ PRNG 算法迭代 D,并将 PRNG 结果放入下一条指令的 S。	2
ZEROX D _i {#}S {WC/WZ/WCZ}	将 D 零扩展至位 S[4:0] 以上。C = 结果的 MSB。*		2

密码和智能密码说明		
操作说明	描述	时钟 齿轮、查找表和轮毂

别针			
迪尔克	{#}D	{WCZ}	引脚 D[10:6]+D[5:0]..D[5:0] 的 DIR 位 = C ₀ 在 DIRA/DIRB 内进行换行。先前的 SETQ 优先于 D[10:6]。C,Z = DIR 少量。
DIRH	{#}D	{WCZ}	引脚 D[10:6]+D[5:0]..D[5:0] 的 DIR 位 = 1 ₀ 包含在 DIRA/DIRB 内。先前的 SETQ 覆盖 D[10:6]。C,Z = DIR 少量。
迪尔	{#}D	{WCZ}	引脚 D[10:6]+D[5:0]..D[5:0] 的 DIR 位 = 0 ₀ 包含在 DIRA/DIRB 内。先前的 SETQ 覆盖 D[10:6]。C,Z = DIR 少量。
DIRNC {#}D		{WCZ}	引脚 D[10:6]+D[5:0]..D[5:0] 的 DIR 位 = !C ₀ 在 DIRA/DIRB 内进行换行。先前的 SETQ 覆盖 D[10:6]。C,Z = DIR 少量。
DIRNOT {#}D		{WCZ}	切换引脚 D[10:6]+D[5:0]..D[5:0] 的 DIR 位。在 DIRA/DIRB 内进行换行。先前的 SETQ 会覆盖 D[10:6]。C,Z = DIR 少量。
DIRNZ {#}D		{WCZ}	引脚 D[10:6]+D[5:0]..D[5:0] 的 DIR 位 = !Z ₀ 在 DIRA/DIRB 内进行换行。先前的 SETQ 覆盖 D[10:6]。C,Z = DIR 少量。
DIRRND {#}D		{WCZ}	引脚 D[10:6]+D[5:0]..D[5:0] 的 DIR 位 = RND ₀ 在 DIRA/DIRB 内进行换行。先前的 SETQ 优先于 D[10:6]。C,Z = DIR 位。
迪尔兹	{#}D	{WCZ}	引脚 D[10:6]+D[5:0]..D[5:0] 的 DIR 位 = Z ₀ 在 DIRA/DIRB 内进行换行。先前的 SETQ 优先于 D[10:6]。C,Z = DIR 少量。
输出/输入选择	{#}D	{WCZ}	引脚 D[10:6]+D[5:0]..D[5:0] 的 OUT 位 = C ₀ DIR 位 = 1 ₀ 在 OUTA/OUTB 内绕回。先前的 SETQ 覆盖 D[10:6]。C,Z = OUT 位。
右心室收缩	{#}D	{WCZ}	引脚 D[10:6]+D[5:0]..D[5:0] 的 OUT 位 = 1 ₀ DIR 位 = 1 ₀ 在 OUTA/OUTB 内绕回。先前的 SETQ 覆盖 D[10:6]。C,Z = OUT 位。
驾驶室容积	{#}D	{WCZ}	引脚 D[10:6]+D[5:0]..D[5:0] 的 OUT 位 = 0 ₀ DIR 位 = 1 ₀ 在 OUTA/OUTB 内绕回。先前的 SETQ 覆盖 D[10:6]。C,Z = OUT 位。
驱动 RVNC {#}D		{WCZ}	引脚 D[10:6]+D[5:0]..D[5:0] 的 OUT 位 = !C ₀ DIR 位 = 1 ₀ 在 OUTA/OUTB 内绕回。先前的 SETQ 覆盖 D[10:6]。C,Z = OUT 位。
DRVNOT {#}D		{WCZ}	切换引脚 D[10:6]+D[5:0]..D[5:0] 的 OUT 位。DIR 位 = 1 ₀ 在 OUTA/OUTB 内绕回。先前的 SETQ 覆盖 D[10:6]。C,Z = OUT 位。
DRVNZ {#}D		{WCZ}	引脚 D[10:6]+D[5:0]..D[5:0] 的 OUT 位 = !Z ₀ DIR 位 = 1 ₀ 在 OUTA/OUTB 内绕回。先前的 SETQ 覆盖 D[10:6]。C,Z = OUT 位。
DRVNRD {#}D		{WCZ}	引脚 D[10:6]+D[5:0]..D[5:0] 的 OUT 位 = RND ₀ DIR 位 = 1 ₀ 在 OUTA/OUTB 内折回。先前的 SETQ 覆盖 D[10:6]。C,Z = OUT 位。
驱动电压源	{#}D	{WCZ}	引脚 D[10:6]+D[5:0]..D[5:0] 的 OUT 位 = Z ₀ DIR 位 = 1 ₀ 在 OUTA/OUTB 内绕回。先前的 SETQ 覆盖 D[10:6]。C,Z = OUT 位。
非逻辑中心	{#}D	{WCZ}	引脚 D[10:6]+D[5:0]..D[5:0] 的 OUT 位 = C ₀ DIR 位 = 0 ₀ 在 OUTA/OUTB 内折回。先前的 SETQ 覆盖 D[10:6]。C,Z = OUT 位。
弗利特	{#}D	{WCZ}	引脚 D[10:6]+D[5:0]..D[5:0] 的 OUT 位 = 1 ₀ DIR 位 = 0 ₀ 在 OUTA/OUTB 内绕回。先前的 SETQ 覆盖 D[10:6]。C,Z = OUT 位。
弗拉基米尔	{#}D	{WCZ}	引脚 D[10:6]+D[5:0]..D[5:0] 的 OUT 位 = 0 ₀ DIR 位 = 0 ₀ 在 OUTA/OUTB 内绕回。先前的 SETQ 覆盖 D[10:6]。C,Z = OUT 位。
FLTNC {#}D		{WCZ}	引脚 D[10:6]+D[5:0]..D[5:0] 的 OUT 位 = !C ₀ DIR 位 = 0 ₀ 在 OUTA/OUTB 内折回。先前的 SETQ 覆盖 D[10:6]。C,Z = OUT 位。
FLTNOT {#}D		{WCZ}	切换引脚 D[10:6]+D[5:0]..D[5:0] 的 OUT 位。DIR 位 = 0 ₀ 在 OUTA/OUTB 内绕回。先前的 SETQ 覆盖 D[10:6]。C,Z = OUT 位。
FLTNZ {#}D		{WCZ}	引脚 D[10:6]+D[5:0]..D[5:0] 的 OUT 位 = !Z ₀ DIR 位 = 0 ₀ 在 OUTA/OUTB 内折回。先前的 SETQ 覆盖 D[10:6]。C,Z = OUT 位。
FLTRND {#}D		{WCZ}	引脚 D[10:6]+D[5:0]..D[5:0] 的 OUT 位 = RND ₀ DIR 位 = 0 ₀ 在 OUTA/OUTB 内折回。先前的 SETQ 覆盖 D[10:6]。C,Z = OUT 位。
弗莱茨	{#}D	{WCZ}	引脚 D[10:6]+D[5:0]..D[5:0] 的 OUT 位 = Z ₀ DIR 位 = 0 ₀ 在 OUTA/OUTB 内折回。先前的 SETQ 覆盖 D[10:6]。C,Z = OUT 位。
非逻辑成员	{#}D	{WCZ}	引脚 D[10:6]+D[5:0]..D[5:0] 的 OUT 位 = C ₀ 在 OUTA/OUTB 内绕回。先前的 SETQ 覆盖 D[10:6]。C,Z = OUT 少量。
南方	{#}D	{WCZ}	引脚 D[10:6]+D[5:0]..D[5:0] 的 OUT 位 = 1 ₀ 在 OUTA/OUTB 内绕回。先前的 SETQ 覆盖 D[10:6]。C,Z = OUT 少量。
出口	{#}D	{WCZ}	引脚 D[10:6]+D[5:0]..D[5:0] 的 OUT 位 = 0 ₀ 在 OUTA/OUTB 内绕回。先前的 SETQ 覆盖 D[10:6]。C,Z = OUT 少量。
OUTNC {#}D		{WCZ}	引脚 D[10:6]+D[5:0]..D[5:0] 的 OUT 位 = !C ₀ 在 OUTA/OUTB 内绕回。先前的 SETQ 覆盖 D[10:6]。C,Z = OUT 少量。
OUTNOT {#}D		{WCZ}	切换引脚 D[10:6]+D[5:0]..D[5:0] 的 OUT 位。在 OUTA/OUTB 内进行换行。先前的 SETQ 会覆盖 D[10:6]。C,Z = 输出位。
OUTNZ {#}D		{WCZ}	引脚 D[10:6]+D[5:0]..D[5:0] 的 OUT 位 = !Z ₀ 在 OUTA/OUTB 内绕回。先前的 SETQ 覆盖 D[10:6]。C,Z = OUT 少量。
OUTRND {#}D		{WCZ}	引脚 D[10:6]+D[5:0]..D[5:0] 的 OUT 位 = RND ₀ 在 OUTA/OUTB 内绕回。先前的 SETQ 优先于 D[10:6]。C,Z = OUT 位。

奥茨 {#}D	{WCZ}	引脚 D[10:6]+D[5:0]..D[5:0] 的 OUT 位 = Z。在 OUTA/OUTB 内回绕。先前的 SETQ 覆盖 D[10:6]。C,Z = OUT 少量。	2
测试P {#}D	WC/WZ测试	引脚 D[5:0] 的 IN 位,写入 C/Z。C/Z = IN[D[5:0]]。	2
测试P {#}D	ORC/ORZ测试	引脚 D[5:0] 的 IN 位,或进入 C/Z。C/Z = C/Z 或 IN[D[5:0]]。	2
测试P {#}D	ANDC/ANDZ测试	引脚 D[5:0] 的 IN 位,并将其与 C/Z 进行与运算。C/Z = C/Z AND IN[D[5:0]]。	2
测试P {#}D	XORC/XORZ测试	引脚 D[5:0] 的 IN 位,将其异或到 C/Z。C/Z = C/Z XOR IN[D[5:0]]。	2
测试PN {#}D	WC/WZ测试	引脚 D[5:0] 的 !IN 位,写入 C/Z。C/Z = !IN[D[5:0]]。	2
测试PN {#}D	ORC/ORZ测试	引脚 D[5:0] 的 !IN 位,或进入 C/Z。C/Z = C/Z OR !IN[D[5:0]]。	2
测试PN {#}D	ANDC/ANDZ测试	引脚 D[5:0] 的 !IN 位,并将其与 C/Z 进行与运算。C/Z = C/Z AND !IN[D[5:0]]。	2
测试PN {#}D	XORC/XORZ测试	引脚 D[5:0] 的 !IN 位,进行异或运算进入 C/Z。C/Z = C/Z XOR !IN[D[5:0]]。	2
智能密码			
AKPIN {#}S		确认智能引脚 S[10:6]+S[5:0]..S[5:0]。包含在 A/B 引脚内。先前的 SETQ 会覆盖 S[10:6]。	2
GETSCP D		将四通道示波器样本放入D中。D = {ch3[7:0],ch2[7:0],ch1[7:0],ch0[7:0]}。	2
RDPIN D,{#}S	{WC}	将智能引脚 S[5:0] 的结果 “Z”读入 D,并确认智能引脚。C = 模态结果。	2
RQPIN D,{#}S	{厕所}	将智能引脚 S[5:0] 的结果 “Z”读入 D,不确认智能引脚 (RQPIN 中的 “Q”表示 “安静”) 。C = 模态结果。	2
SETDACS {#}D		DAC3 = D[31:24],DAC2 = D[23:16],DAC1 = D[15:8],DAC0 = D[7:0]。	2
SETSCP {#}D		将四通道示波器使能设置为D[6],并将输入引脚基极设置为D[5:2]。	2
WRPIN {#}D,{#}S		将智能引脚 S[10:6]+S[5:0]..S[5:0] 的模式设置为 D,并确认智能引脚。包含在 A/B 引脚内。优先于 SETQ 覆盖 S[10:6]。	2
WXPIN {#}D,{#}S		将智能引脚 S[10:6]+S[5:0]..S[5:0] 中的 “X”设置为 D,确认智能引脚。包含在 A/B 引脚内。优先 SETQ 覆盖 S[10:6]。	2
WYPIN {#}D,{#}S		将智能引脚 S[10:6]+S[5:0]..S[5:0] 的 “Y”设置为 D,确认智能引脚。包含在 A/B 引脚内。优先 SETQ 覆盖 S[10:6]。	2

分支指令		
操作说明	描述	时钟 齿轮和查找表/集线器
称呼 {#}\一个	通过将 {C, Z, 10 b0, PC[19:0]} 压栈来调用 A。如果 R = 1,则 PC += A,否则 PC = A。 “\”强制 R = 0。	4 / 13...20
称呼 D {WC/WZ/WCZ}	通过将 C, Z, 10 b0, PC[19:0]} 压入堆栈来调用 D。C = D[31],Z = D[30],PC = D[19:0]。	4 / 13...20
马蹄莲 {#}\A	通过在 PTRB++ 处将 {C, Z, 10 b0, PC[19:0]} 写入集线器 long 来调用 A。如果 R = 1,则 PC += A,否则 PC = A。 “\”强制 R = 0。	5...12 ¹ / 14...32 ¹
马蹄莲 {WC/WZ/WCZ}	通过在 PTRB++ 时将 {C, Z, 10 b0, PC[19:0]} 写入集线器来调用 D。C = D[31],Z = D[30],PC = D[19:0]。	5...12 ¹ / 14...32 ¹
呼叫 {#}\A	通过在 PTRB++ 处向集线器写入 {C, Z, 10 b0, PC[19:0]} 来调用 A。如果 R = 1,则 PC += A,否则 PC = A。 “\”强制 R = 0。	5...12 ¹ / 14...32 ¹
呼叫D {WC/WZ/WCZ}	通过在 PTRB++ 时将 {C, Z, 10 b0, PC[19:0]} 写入集线器长整型来调用 D。C = D[31],Z = D[30],PC = D[19:0]。	5...12 ¹ / 14...32 ¹
CALLD D,{#}S {WC/WZ/WCZ}	通过将 {C, Z, 10 b0, PC[19:0]} 写入 D 来调用 S**。C = S[31],Z = S[30]。	4 / 13...20
呼叫 PA/PB/PTRA/PTRB,{#}\A	通过将 {C, Z, 10 b0, PC[19:0]} 写入 PA/PB/PTRA/PTRB (每个W)来调用 A。如果 R = 1,则 PC += A,否则 PC = A。 “\”力 R = 0。	4 / 13...20
卡尔帕 {#}D,{#}S	通过将 {C, Z, 10 b0, PC[19:0]} 推送到堆栈来调用 S** ,并将 D 复制到 PA。	4 / 13...20
调用PB {#}D,{#}S	通过将 {C, Z, 10 b0, PC[19:0]} 推送到堆栈来调用 S** ,将 D 复制到 PB。	4 / 13...20
东南风 D,{#}S	如果结果为 \$FFFF_FFFF,则减少 D 并跳转到 S**。	2 或 4 / 2 或 13...20
丹尼森 D,{#}S	如果结果不是 \$FFFF_FFFF,则减少 D 并跳转到 S**。	2 或 4 / 2 或 13...20
DJNZ D,{#}S	如果结果不为零,则减少 D 并跳转到 S**。	2 或 4 / 2 或 13...20
DJZ D,{#}S	如果结果为零,则减少 D 并跳转到 S**。	2 或 4 / 2 或 13...20
EXECF {#}D	跳转到 cog/LUT 中的 D[9:0],并将 SKIPF 模式设置为 D[31:10]。PC = {10 b0, D[9:0]}。	4 / 4
新西兰 D,{#}S	如果结果不为零,则增加 D 并跳转到 S**。	2 或 4 / 2 或 13...20
伊吉兹 D,{#}S	如果结果为零,则增加 D 并跳转到 S**。	2 或 4 / 2 或 13...20
跳转 {#}\一个	跳转到 A。如果 R = 1 则 PC += A,否则 PC = A。 “\”强制 R = 0。	4 / 13...20
跳转 D {WC/WZ/WCZ}	D。C = D[31],Z = D[30],PC = D[19:0]。	4 / 13...20
JMPREL {#}D	使用 D 指令向前/向后跳转。对于 cogex,PC += D[19:0]。对于 hubex,PC += D[17:0] << 2。	4 / 13...20
答复 {#}D,{#}S	执行接下来的 D[8:0] 指令 S 次。如果 S = 0,则无限次重复指令。如果 D[8:0] = 0,则不重复任何指令。	2 / 2

RESI0	从 INT0 恢复。(CALLD \$1FE,\$1FF WCZ)	4 / 13...20
RESI1	从 INT1 恢复。(CALLD \$1F4,\$1F5 WCZ)	4 / 13...20
RESI2	从 INT2 恢复。(CALLD \$1F2,\$1F3 WCZ)	4 / 13...20
RESI3	从 INT3 恢复。(CALLD \$1F0,\$1F1 WCZ)	4 / 13...20
RET	{WC/WZ/WCZ}通过弹出堆栈返回 (K),C = K[31],Z = K[30],PC = K[19:0]。	4 / 13...20
雷塔	{WC/WZ/WCZ}通过读取 -PTRA 处的集线器长度 (L) 返回,C = L[31],Z = L[30],PC = L[19:0]。	11...18 ¹ / 20...40 ¹
可再编程技术	{WC/WZ/WCZ}通过在 -PTRB 处读取集线器长度 (L) 返回,C = L[31],Z = L[30],PC = L[19:0]。	11...18 ¹ / 20...40 ¹
RET0	从 INT0 返回。(CALLD \$1FF,\$1FF WCZ)	4 / 13...20
雷蒂1	从 INT1 返回。(CALLD \$1FF,\$1F5 WCZ)	4 / 13...20
雷蒂2	从 INT2 返回。(CALLD \$1FF,\$1F3 WCZ)	4 / 13...20
雷蒂3	从 INT3 返回。(CALLD \$1FF,\$1F1 WCZ)	4 / 13...20
跳过 {#}D	每个 D 跳过指令,后续指令 0..31 会针对 D[0]..D[31] 中的每个 “1”位取消。	2 / 2
SKIPF {#}D	按照 D 快速跳过 cog/LUT 指令。类似于 SKIP,但 PC 不会取消指令,而是跳过它们。	2 / 非法
特杰夫 D,{#}S	测试 D 并且如果 D 已满 (D = \$FFFF_FFFF) 则跳转到 S**。	2 或 4 / 2 或 13...20
特杰夫 D,{#}S	测试 D 并且如果 D 未滿 (D != \$FFFF_FFFF) 则跳转到 S**。	2 或 4 / 2 或 13...20
天津市 D,{#}S	测试 D,如果 D 无符号 (D[31] = 0) ,则跳转到 S**。	2 或 4 / 2 或 13...20
新西兰 D,{#}S	测试 D,如果 D 不为零,则跳转到 S**。	2 或 4 / 2 或 13...20
泰杰斯 D,{#}S	测试 D 并且如果 D 有符号 (D[31] = 1)则跳转到 S**。	2 或 4 / 2 或 13...20
谷歌公司 D,{#}S	测试 D,如果 D 溢出 (D[31] != C,C = 上次加法/减法得出的 “正确符号”) ,则跳转到 S**。	2 或 4 / 2 或 13...20
泰吉中 D,{#}S	测试 D,如果 D 为零,则跳转到 S**。	2 或 4 / 2 或 13...20

¹ 如果跨越枢纽较长,则 +1

集线器控制、FIFO 和 RAM 指令		
操作说明	描述	时钟 齿轮和查找表/集线器
集线器控制		
COGID {#}D	{WC}如果 D 是寄存器且没有 WC,则将 cog ID (0 到 15)放入 D。如果是 WC,则检查 cog D[3:0] 的状态,如果打开则 C = 1。	2...9,如果结果/相同则 +2
认知 {#}D,{#}S	{WC}启动由 D 选择的 cog,S[19:0] 设置集线器启动地址和 cog 的 PTRB。之前的 SETQ 设置 cog 的 PTRA。	2...9,如果结果/相同则 +2
齿轮停止 {#}D	停止齿轮D[3:0]。	2...9 / 相同
洛克纽德	{WC}请求一个 LOCK,D 将被写入 LOCK 编号 (0 到 15) 。如果没有可用的 LOCK,则 C = 1。	4...11 / 相同
锁定REL {#}D	{厕所}释放 LOCK D[3:0]。如果 D 是寄存器和 WC,则获取 LOCK 所有者的当前/最后一个 cog id,使其进入 D 和 LOCK 状态进入 C。	2...9,如果结果/相同则 +2
锁定RET {#}D	返回 LOCK D[3:0] 以进行重新分配。	2...9 / 相同
锁具 {#}D	{厕所}尝试获取 LOCK D[3:0]。如果获取到 LOCK,则 C = 1。LOCKREL 释放 LOCK。如果所有者的齿轮停止或重新启动。	2...9,如果结果/相同则 +2
集线器 {#}D	将集线器配置设置为 D。	2...9 / 相同
集线器先进先出		
GETPTR D	将当前 FIFO 集线器指针放入 D。	2 / 先进先出
FBLOCK {#}D,{#}S	设置块换行时的下一个块。D[13:0] = 块大小 (以 64 字节为单位) (0 = 最大) ,S[19:0] = 块起始地址。	2 / 先进先出
RDFAST {#}D,{#}S	通过 FIFO 开始新的快速集线器读取。D[31] = 无需等待,D[13:0] = 块大小以 64 字节为单位 (0 = 最大) ,S[19:0] = 块起始地址。	2 或 WRFAST 完成 + 10...17 / FIFO 正在使用
WRFAST {#}D,{#}S	通过 FIFO 开始新的快速集线器写入。D[31] = 无需等待,D[13:0] = 块大小以 64 字节为单位 (0 = 最大) ,S[19:0] = 块起始地址。	2 或 WRFAST 完成 + 3 / 先进先出正在使用
RFBYTE D	{WC/WZ/WCZ}在 RDFAST 之后使用。将零扩展字节从 FIFO 读入 D,C = 字节的 MSB。*	2 / 先进先出
射频D	{WC/WZ/WCZ}在 RDFAST 之后使用。将 long 从 FIFO 读取到 D,C = long 的 MSB。*	2 / 先进先出
射频无功补偿	{WC/WZ/WCZ}在 RDFAST 之后使用。将零扩展的 1.4 字节值从 FIFO 读取到 D,C = 0。*	2 / 先进先出
射频无功功率比	{WC/WZ/WCZ}在 RDFAST 之后使用。将符号扩展的 1.4 字节值从 FIFO 读取到 D,C = 值的 MSB。*	2 / 先进先出
射频字	{WC/WZ/WCZ}在 RDFAST 之后使用。将零扩展字从 FIFO 读入 D,C = 字的 MSB。*	2 / 先进先出
WFBYTE {#}D	在WRFAST之后使用。将D[7:0]中的字节写入FIFO。	2 / 先进先出
WFLONG {#}D	在 WRFAST 之后使用。将 D[31:0] 中的 long 写入 FIFO。	2 / 先进先出
WFWORD {#}D	在 WRFAST 之后使用。将 D[15:0] 中的字写入 FIFO。	2 / 先进先出

集线器内存		
波帕 D {WC/WZ/WCZ}	从集线器地址--PTRA 读取长整型数据到 D. C = 长整型的 MSB。*	9...16 ¹ / 9...26 ¹
聚四氯乙烯 D {WC/WZ/WCZ}	从集线器地址--PTRB 读取长整型数据到 D. C = 长整型的 MSB。*	9...16 ¹ / 9...26 ¹
RDBYTE D,{#}S/P {WC/WZ/WCZ}	将集线器地址 {#}S/PTRx 中的零扩展字节读取到 D. C = 字节的 MSB。 *	9...16 / 9...26
RDLONG D,{#}S/P {WC/WZ/WCZ}	将集线器地址 {#}S/PTRx 中的 long 读入 D.C = long 的 MSB。* 先前的 SETQ/SETQ2 调用 cog/LUT 块转移。	9...16 ¹ / 9...26 ¹
RDWORD D,{#}S/P {WC/WZ/WCZ}	将集线器地址 {#}S/PTRx 中的零扩展字读入 D.C = 字的 MSB。*	9...16 ¹ / 9...26 ¹
PUSHA {#}D	将 D[31:0] 中的长整型写入集线器地址 PTRA++。	3...10 ¹ / 3...20 ¹
推HB {#}D	将 D[31:0] 中的长整型写入集线器地址 PTRB++。	3...10 ¹ / 3...20 ¹
WMLONG D,{#}S/P	将 D[31:0] 中的非 \$00 字节写入集线器地址 {#}S/PTRx。之前的 SETQ/SETQ2 调用 cog/LUT 模块转移。	3...10 ¹ / 3...20 ¹
写字节 {#}D,{#}S/P	将 D[7:0] 中的字节写入集线器地址 {#}S/PTRx。	3...10 / 3...20
WRLONG {#}D,{#}S/P	将 D[31:0] 中的长整型写入集线器地址 {#}S/PTRx。之前的 SETQ/SETQ2 调用 cog/LUT 块传输。	3...10 ¹ / 3...20 ¹
写字 {#}D,{#}S/P	将 D[15:0] 中的字写入集线器地址 {#}S/PTRx。	3...10 ¹ / 3...20 ¹

¹ 如果跨越枢纽较长,则 +1

活动说明		
操作说明	描述	时钟 齿轮和查找表/集线器
ADDCT1 D,{#}S	将 CT1 事件设置为在 CT = D + S 时触发,将 S 加到 D 中。	2
ADDCT2 D,{#}S	将 CT2 事件设置为在 CT = D + S 时触发,将 S 加到 D 中。	2
ADDCT3 D,{#}S	设置 CT3 事件在 CT = D + S 时触发,将 S 加到 D 中。	2
COGATN {#}D	选通 “注意”所有在 D[15:0] 中对应位较高的齿轮。	2
JATN {#}秒	如果设置了 ATN 事件标志,则跳转到 S**。	2 或 4 / 2 或 13...20
JCT1 {#}秒	如果设置了 CT1 事件标志,则跳转到 S**。	2 或 4 / 2 或 13...20
联合CT2 {#}秒	如果设置了 CT2 事件标志,则跳转到 S**。	2 或 4 / 2 或 13...20
JCT3 {#}秒	如果设置了 CT3 事件标志,则跳转到 S**。	2 或 4 / 2 或 13...20
联合开发子弹 {#}秒	如果设置了 FBW 事件标志,则跳转到 S**。	2 或 4 / 2 或 13...20
吉特 {#}秒	如果设置了 INT 事件标志,则跳转到 S**。	2 或 4 / 2 或 13...20
JNATN {#}S	如果 ATN 事件标志已清除,则跳转到 S**。	2 或 4 / 2 或 13...20
JNCT1 {#}S	如果 CT1 事件标志已清除,则跳转到 S**。	2 或 4 / 2 或 13...20
JNCT2 {#}S	如果 CT2 事件标志已清除,则跳转到 S**。	2 或 4 / 2 或 13...20
JNCT3 {#}S	如果 CT3 事件标志已清除,则跳转到 S**。	2 或 4 / 2 或 13...20
JNFBW {#}S	如果 FBW 事件标志已清除,则跳转到 S**。	2 或 4 / 2 或 13...20
JNINT {#}S	如果 INT 事件标志已清除,则跳转到 S**。	2 或 4 / 2 或 13...20
JNPAT {#}S	如果 PAT 事件标志已清除,则跳转到 S**。	2 或 4 / 2 或 13...20
JNQMT {#}S	如果 QMT 事件标志已清除,则跳转到 S**。	2 或 4 / 2 或 13...20
JNSE1 {#}S	如果 SE1 事件标志已清除,则跳转到 S**。	2 或 4 / 2 或 13...20
JNSE2 {#}S	如果 SE2 事件标志已清除,则跳转到 S**。	2 或 4 / 2 或 13...20
JNSE3 {#}S	如果 SE3 事件标志已清除,则跳转到 S**。	2 或 4 / 2 或 13...20
JNSE4 {#}S	如果 SE4 事件标志已清除,则跳转到 S**。	2 或 4 / 2 或 13...20
JNXFI {#}S	如果 XF1 事件标志已清除,则跳转到 S**。	2 或 4 / 2 或 13...20
JNXMT {#}S	如果 XMT 事件标志已清除,则跳转到 S**。	2 或 4 / 2 或 13...20
JNXRL {#}S	如果 XRL 事件标志已清除,则跳转到 S**。	2 或 4 / 2 或 13...20
JNXRO {#}S	如果 XRO 事件标志已清除,则跳转到 S**。	2 或 4 / 2 或 13...20
JPAT {#}秒	如果设置了 PAT 事件标志,则跳转到 S**。	2 或 4 / 2 或 13...20
吉庆矿业 {#}秒	如果设置了 QMT 事件标志,则跳转到 S**。	2 或 4 / 2 或 13...20
JSE1 {#}秒	如果设置了 SE1 事件标志,则跳转到 S**。	2 或 4 / 2 或 13...20
JSE2 {#}秒	如果设置了 SE2 事件标志,则跳转到 S**。	2 或 4 / 2 或 13...20
JSE3 {#}秒	如果设置了 SE3 事件标志,则跳转到 S**。	2 或 4 / 2 或 13...20
JSE4 {#}秒	如果设置了 SE4 事件标志,则跳转到 S**。	2 或 4 / 2 或 13...20
江西金融 {#}秒	如果设置了 XF1 事件标志,则跳转到 S**。	2 或 4 / 2 或 13...20

江西移动	{#}秒	如果设置了 XMT 事件标志,则跳转到 S**。	2 或 4 / 2 或 13...20
江西铁路	{#}秒	如果设置了 XRL 事件标志,则跳转到 S**。	2 或 4 / 2 或 13...20
杰西罗	{#}秒	如果设置了 XRO 事件标志,则跳转到 S**。	2 或 4 / 2 或 13...20
污染源	{WC/WZ/WCZ}将 ATN	事件标志放入 C/Z,然后清除它。	2
投票CT1	{WC/WZ/WCZ}将 CT1	事件标志放入 C/Z,然后清除它。	2
投票2	{WC/WZ/WCZ}将 CT2	事件标志放入 C/Z,然后清除它。	2
投票CT3	{WC/WZ/WCZ}将 CT3	事件标志放入 C/Z,然后清除它。	2
波拉波夫	{WC/WZ/WCZ}将 FBW	事件标志放入 C/Z,然后清除它。	2
民意调查	{WC/WZ/WCZ}将 INT	事件标志放入 C/Z,然后清除它。	2
波帕特	{WC/WZ/WCZ}将 PAT	事件标志放入 C/Z,然后清除它。	2
民意调查	{WC/WZ/WCZ}将 QMT	事件标志放入 C/Z,然后清除它。	2
民意调查1	{WC/WZ/WCZ}将 SE1	事件标志放入 C/Z,然后清除它。	2
民意调查2	{WC/WZ/WCZ}将 SE2	事件标志放入 C/Z,然后清除它。	2
民意调查3	{WC/WZ/WCZ}将 SE3	事件标志放入 C/Z,然后清除它。	2
POLLSE4	{WC/WZ/WCZ}将 SE4	事件标志放入 C/Z,然后清除它。	2
波利克斯菲	{WC/WZ/WCZ}将 XF1	事件标志放入 C/Z,然后清除它。	2
轮询	{WC/WZ/WCZ}将 XMT	事件标志放入 C/Z,然后清除它。	2
轮询XRL	{WC/WZ/WCZ}将 XRL	事件标志放入 C/Z,然后清除它。	2
波克斯罗	{WC/WZ/WCZ}将 XRO	事件标志放入 C/Z,然后清除它。	2
设置{#}D,{#}S		设置 PAT 事件的引脚模式。C 选择 IN/A/INB,Z 选择 =/!=,D 提供掩码值,S 提供匹配值。	2
SETSE1 {#}D		将SE1事件配置设置为D[8:0]。	2
SETSE2 {#}D		将SE2事件配置设置为D[8:0]。	2
SETSE3 {#}D		将SE3事件配置设置为D[8:0]。	2
SETSE4 {#}D		将SE4事件配置设置为D[8:0]。	2
等待	{WC/WZ/WCZ}等待 ATN	事件标志,然后清除它。之前的 SETQ 设置可选的 CT 超时值。C/Z = 超时。	2+
等待CT1	{WC/WZ/WCZ}等待 CT1	事件标志,然后清除它。之前的 SETQ 设置可选的 CT 超时值。C/Z = 超时。	2+
等待CT2	{WC/WZ/WCZ}等待 CT2	事件标志,然后清除它。之前的 SETQ 设置可选的 CT 超时值。C/Z = 超时。	2+
等待CT3	{WC/WZ/WCZ}等待 CT3	事件标志,然后清除它。之前的 SETQ 设置可选的 CT 超时值。C/Z = 超时。	2+
等待FBW	{WC/WZ/WCZ}等待 FBW	事件标志,然后清除它。之前的 SETQ 设置可选的 CT 超时值。C/Z = 超时。	2+
等待时间	{WC/WZ/WCZ}等待 INT	事件标志,然后清除它。之前的 SETQ 设置可选的 CT 超时值。C/Z = 超时。	2+
韦特帕特	{WC/WZ/WCZ}等待 PAT	事件标志,然后清除它。之前的 SETQ 设置可选的 CT 超时值。C/Z = 超时。	2+
WAITSE1	{WC/WZ/WCZ}等待 SE1	事件标志,然后清除它。之前的 SETQ 设置可选的 CT 超时值。C/Z = 超时。	2+
WAITSE2	{WC/WZ/WCZ}等待 SE2	事件标志,然后清除它。之前的 SETQ 设置可选的 CT 超时值。C/Z = 超时。	2+
WAITSE3	{WC/WZ/WCZ}等待 SE3	事件标志,然后清除它。之前的 SETQ 设置可选的 CT 超时值。C/Z = 超时。	2+
WAITSE4	{WC/WZ/WCZ}等待 SE4	事件标志,然后清除它。之前的 SETQ 设置可选的 CT 超时值。C/Z = 超时。	2+
等待	{WC/WZ/WCZ}等待 XF1	事件标志,然后清除它。之前的 SETQ 设置可选的 CT 超时值。C/Z = 超时。	2+
等待	{WC/WZ/WCZ}等待 XMT	事件标志,然后清除它。之前的 SETQ 设置可选的 CT 超时值。C/Z = 超时。	2+
等待TXRL	{WC/WZ/WCZ}等待 XRL	事件标志,然后清除它。之前的 SETQ 设置可选的 CT 超时值。C/Z = 超时。	2+
威特克斯罗	{WC/WZ/WCZ}等待 XRO	事件标志,然后清除它。之前的 SETQ 设置可选的 CT 超时值。C/Z = 超时。	2+

中断指令		
操作说明	描述	时钟 齿轮、查找表和轮毂
允许	允许中断（默认）。	2
BRK {#}D	如果在调试 ISR 中,则将下一个中断条件设置为 D。否则,将 BRK 代码设置为 D[7:0] 并无条件触发 BRK 中断（如果启用）。	2
COGBRK {#}D	如果在调试ISR中,则在cog D[3:0]中触发异步断点。Cog D[3:0]必须具有异步断点。已启用。	2
GETBRK D	WC/WZ/WCZ根据 WC/WZ/WCZ 将断点/齿轮状态写入 D。详情请参阅文档。	2
NIXINT1	取消 INT1。	2
NIXINT2	取消 INT2。	2

NIXINT3	取消 INT3。	2
SETINT1 {#}D	将INT1源设置为D[3:0]。	2
SETINT2 {#}D	将INT2源设置为D[3:0]。	2
SETINT3 {#}D	将INT3源设置为D[3:0]。	2
斯塔利	停止中断。	2
TRGINT1	触发 INT1,无论 STALLI 模式如何。	2
TRGINT2	触发 INT2,无论 STALLI 模式如何。	2
TRGINT3	触发 INT3,无论 STALLI 模式如何。	2

寄存器间接指令		
操作说明	描述	时钟 齿轮和查找表/集线器
ALTB D, _i {#}S	将下一条指令的D字段更改为D[13:5]。	2
ALTB D, _i {#}S	将下一条指令的 D 字段更改为 (D[13:5] + S) & \$1FF,D += 符号扩展的 S[17:9]。	2
阿尔特达 D	将下一条指令的D字段改为D[8:0]。	2
阿尔特达 D, _i {#}S	将下一条指令的 D 字段更改为 (D + S) & \$1FF,D += 符号扩展的 S[17:9]。	2
阿尔特GB D	修改后续的 GETBYTE/ROLBYTE 指令。下一个 S 字段 = D[10:2],N 字段 = D[1:0]。	2
ALTGB D, _i {#}S	修改后续的 GETBYTE/ROLBYTE 指令。下一个 S 字段 = (D[10:2] + S) & \$1FF,N 字段 = D[1:0],D += 符号扩展的 S[17:9]。	2
备用地	修改后续的 GETNIB/ROLNIB 指令。下一个 S 字段 = D[11:3],N 字段 = D[2:0]。	2
备选方案 D, _i {#}S	修改后续 GETNIB/ROLNIB 指令。下一个 S 字段 = (D[11:3] + S) & \$1FF,N 字段 = D[2:0],D += 符号扩展的 S[17:9]。	2
阿尔特格	修改后续的 GETWORD/ROLWORD 指令。下一个 S 字段 = D[9:1],N 字段 = D[0]。	2
ALTGW D, _i {#}S	修改后续的 GETWORD/ROLWORD 指令。下一个 S 字段 = ((D[9:1] + S) & \$1FF),N 字段 = D[0],D += 符号扩展的 S[17:9]。	2
阿尔蒂 D	执行 D 来代替下一条指令。D 保持不变。	2
阿尔蒂 D, _i {#}S	根据 S _i 用 D 中的字段替换下一条指令的 I/R/D/S 字段。根据 S 修改 D。	2
ALTR D	将下一条指令的结果寄存器地址 (通常为D域)修改为D[8:0]。	2
ALTR D, _i {#}S	将下一条指令的结果寄存器地址 (通常为 D 字段)更改为 (D + S) & \$1FF,D += 符号扩展的 S[17:9]。	2
阿尔特斯 D	将下一条指令的S字段改为D[8:0]。	2
阿尔特斯 D, _i {#}S	将下一条指令的 S 字段更改为 (D + S) & \$1FF,D += 符号扩展的 S[17:9]。	2
ALTSB D	修改后续 SETBYTE 指令。下一个 D 字段 = D[10:2],N 字段 = D[1:0]。	2
ALTSB D, _i {#}S	修改后续 SETBYTE 指令。下一个 D 字段 = (D[10:2] + S) & \$1FF,N 字段 = D[1:0],D += 符号扩展 S[17:9]。	2
阿尔特森	修改后续 SETNIB 指令。下一个 D 字段 = D[11:3],N 字段 = D[2:0]。	2
ALTSN D, _i {#}S	修改后续 SETNIB 指令。下一个 D 字段 = (D[11:3] + S) & \$1FF,N 字段 = D[2:0],D += 符号扩展 S[17:9]。	2
阿尔茨海姆	修改后续 SETWORD 指令。下一个 D 字段 = D[9:1],N 字段 = D[0]。	2
ALTSW D, _i {#}S	修改后续 SETWORD 指令。下一个 D 字段 = (D[9:1] + S) & \$1FF,N 字段 = D[0],D += 符号扩展 S[17:9]。	2

CORDIC 求解器说明		
操作说明	描述	时钟 齿轮、查找表和轮毂
GETQX D {WC/WZ/WCZ}将 CORDIC 结果 X 检索到 D 中。如果结果尚未准备好,则等待。C = X[31]。	¹	2...58
GETQY D {WC/WZ/WCZ}将 CORDIC 结果 Y 检索到 D 中。如果结果尚未准备好,则等待。C = Y[31]。	¹	2...58
季度分期 {#}D, _i {#}S	开始 CORDIC 无符号除法 [SETQ 值或 32 - b ₀ , D] / S _i 。GETQX/GETQY 检索商/余数。	2...9
速递 {#}D	开始对 D 进行 CORDIC 对数到数字的转换。GETQX 检索数字。	2...9
QFRAC {#}D, _i {#}S	开始 CORDIC 无符号除法 [D, SETQ 值或 32 - b ₀] / S _i 。GETQX/GETQY 检索商/余数。	2...9
质量日志 {#}D	开始对 D 进行 CORDIC 数字到对数的转换。GETQX 检索对数[5 - whole_exponent, 2 ⁷ - 分数指数]。	2...9
位级乘法或除法 {#}D, _i {#}S	开始 D * S 的 CORDIC 无符号乘法。GETQX/GETQY 检索下限/上限乘积。	2...9

QROTATE {#}D _i {#}S	开始按角度 S 对点 (D,SETQ 值或 32 - b0) 进行 CORDIC 旋转。GETQX/GETQY 检索 X/Y。	2...9
QSQRT {#}D _i {#}S	开始 CORDIC 计算 {S, D} 的平方根。GETQX 检索根。	2...9
QVECTOR {#}D _i {#}S	开始点 (D, S) 的 CORDIC 矢量化。GETQX/GETQY 检索长度/角度。	2...9

¹ Z = (结果 == 0)

色彩空间转换器和像素混合器说明		
操作说明	描述	时钟 齿轮、查找表和轮毂
色彩空间转换器		
设置TCFRQ {#}D	将色彩空间转换器 “CFRQ”参数设置为D[31:0]。	2
SETCI {#}D	将色彩空间转换器 “CI”参数设置为 D[31:0]。	2
设置模式 {#}D	将色彩空间转换器 “CMOD”参数设置为D[8:0]。	2
设置TCQ {#}D	将色彩空间转换器 “CQ”参数设置为D[31:0]。	2
SETCY {#}D	将色彩空间转换器 “CY”参数设置为 D[31:0]。	2
像素混合器		
ADDPiX D _i {#}S	将 S 的字节添加到 D 的字节中,并使用 \$FF 饱和度。	7
BLNPiX D _i {#}S	使用 SETPIV 值将 S 的字节 Alpha 混合到 D 的字节中。	7
MIXPiX D _i {#}S	使用 SETPIX 和 SETPIV 值将 S 的字节混合到 D 的字节中。	7
MULPiX D _i {#}S	将 S 的字节乘以 D 的字节,其中 \$FF = 1.0 且 \$00 = 0.0。	7
设定IV {#}D	将 BLNPiX/MIXPiX 混合因子设置为 D[7:0]。	2
设置 IX {#}D	将MIXPiX模式设置为D[5:0]。	2

查找表、流器和其他指令		
操作说明	描述	时钟 齿轮和查找表/集线器
查找表		
RD LUT D _i {#}S/P {WC/WZ/WCZ}	将数据从 LUT 地址 {#}S/PTRx 读取到 D _i C = 数据的 MSB。*	3
SETLUTS {#}D	如果 D[0] = 1,则启用 LUT 共享,其中复制相邻奇数/偶数伴随 cog 内的 LUT 写入到这个齿轮的 LUT。	2
WRLUT {#}D _i {#}S/P	将 D 写入 LUT 地址 {#}S/PTRx。	2
飘带		
GETXACC D	将流光器的 Goertzel X 累加器放入 D,将 Y 累加器放入下一条指令的 S,清除累加器。	2
SETXFRQ {#}D	将流光 NCO 频率设置为 D。	2
XCONT {#}D _i {#}S	缓冲在当前命令的最终 NCO 翻转时发出的新流光命令,继续阶段。	2+
XINIT {#}D _i {#}S	立即发出流光指令,归零阶段。	2
XSTOP	立即停止流媒体。	2
XZERO {#}D _i {#}S	缓冲新的流光命令,该命令将在当前命令的最终 NCO 翻转,归零阶段发出。	2+
各种各样的		
澳大利亚 #n	队列 #n 将用作下一个 #D 出现的高 23 位,因此下一个 9 位 #D 将增加到 32 位。	2
奥格斯 #n	队列 #n 将用作下一个 #S 出现的高 23 位,因此下一个 9 位 #S 将增加到 32 位。	2
GETCT D {厕所}	如果 WC 进入 D,则获取 CT[31:0] 或 CT[63:32]。GETCT WC + GETCT 获得完整 CT。复位时 CT=0,每个时钟周期 CT++。C = 相同的。	2
GETRND WC/WZ/WCZ将 RND 放入 C/Z。C = RND[31],Z = RND[30],每个齿轮独一无二。		2
获得 D {WC/WZ/WCZ}	将 RND 放入 D/C/Z 中,RND 是每个时钟都会更新的伪随机数生成器 (PRNG)。D = RND[31:0],C = RND[31],Z = RND[30],每个齿轮都是唯一的。	2
NOP	无操作。	2
流行音乐 D {WC/WZ/WCZ}弹出堆栈 (K)。D = K。C = K[31]。*		2
推 {#}D	将 D 推入堆栈。	2

SETQ {#}D	将 Q 设置为 D。在 RDLONG/WRLONG/WMLONG 之前使用以设置块传输。也可用于之前 MUXQ/COGINIT/QDIV/QFRAC/QROTATE/WAITxxx。	2
SETQ2 {#}D	将 Q 设置为 D。在 RDLONG/WRLONG/WMLONG 之前使用来设置 LUT 块传输。	2
等待 {#}D	{WC/WZ/WCZ}如果没有 WC/WZ/WCZ,则等待 2 + D 个时钟周期。如果有 WC/WZ/WCZ,则等待 2 + (D & RND) 个时钟周期。C/Z = 0。	2 + D

系统特性

绝对最大电气额定值

超过绝对最大额定值的应力可能会对器件造成永久性损坏。这些是绝对应力额定值。器件的功能操作并不暗示在这些或任何其他条件下超过给定值。长时间暴露在绝对最大额定值下可能会对器件产生不利影响可靠性。

绝对最大额定值	
偏置条件下的环境温度	-40 °C 至 +125 °C
储存温度	-40 °C 至 +150 °C
VDD 相对于 GND 的电压	-0.3 V 至 +2.2 V
Vxxyy 相对于 GND 的电压	-0.3 V 至 +4.0 V
所有其他引脚相对于 GND1 的电压	-0.3 V 至 (Vxxyy + 0.3 V)
总功率耗散	2.5 瓦
最大流出 GND 的电流	4 A
流入 VDD 引脚的最大电流	每针 120 毫安
流入 Vxxyy 引脚的最大电流	每针 120 毫安
最大直流电流进入具有内部保护二极管正向偏置的输入引脚	±10 毫安
每个I/O引脚的最大允许电流	±30 毫安
ESD人体模型 (JS-001)	4千伏
ESD充电器件模型 (JS-002)	1千伏

¹ 注意 :如果内部保护二极管正向偏置电流为未超过。

直流特性

工作温度范围：-40°C 至 +105°C（除非另有说明）。

直流特性						
符号参数		状况	分钟	类型1	最大限度	单位
电压源	核心电源电压		1.7	1.8	1.9	五
维	VIO 电源电压		3.15	3.3	3.45	五
维赫	输入逻辑阈值		维 * 0.3	维 * 0.5	维 * 0.7伏	
..	输入漏电流	IO 引脚Vin = GND 或Vio		±0.1	±10	µA

卷	输出低电压（相对于GND）	VDD=3.3V,吸收 1mA VDD=3.3V,吸收 10mA VDD=3.3V,吸收 30mA		15 160 510		毫伏
沃	输出高电压（相对于Vxxyy）	VDD=3.3V,拉电流 1mA VDD=3.3V,拉电流 10mA VDD=3.3V,拉电流 30mA		-6 -170 -580		毫伏
Iq Vdd VDD 静态 当前的		RESn = 测试 = P0..P64 = 0V,Vxxyy = 3.3V,VDD = 1.8V		40		μA
Iq Vxxyy Vxxyy 静止 当前的		RESn = 测试 = P0..P64 = 0V,Vxxyy = 3.3V,VDD = 1.8V		0.5		μA

¹ 注:除非另有说明,典型值 “Typ”列中的数据为T = 25 °C。

交流特性工作温度范围：-40°C
至 +105°C,除非另有说明。

交流特性						
象征	范围	状况	分钟	典型值 ¹	最大单位	
频率	振荡器频率 RCSLOW（内部）	RCFAST（内部） 直接驱动（进入 XI） 水晶（XI 和 XO 之间） PLL（由直接驱动器或晶体供电）	12 20 直 流 1 3.33	20 24 - - 2 180	30 30 200 50 320	千赫 兆赫 兆赫 兆赫 兆赫
辛	XI 和 XO 引脚 电容	模式 0:禁用（1MΩ 反馈电阻关闭） 模式1:直接驱动 模式 2:晶振≥16MHz 模式 3:晶振 < 16MHz		2 2 15 30		皮法 皮法 皮法 皮法

¹ 除非另有说明,典型值 “Typ”列中的数据为T = 25 °C。

² 标称 PLL 频率（系统时钟速度)在最高 105°C 时为 180 MHz。

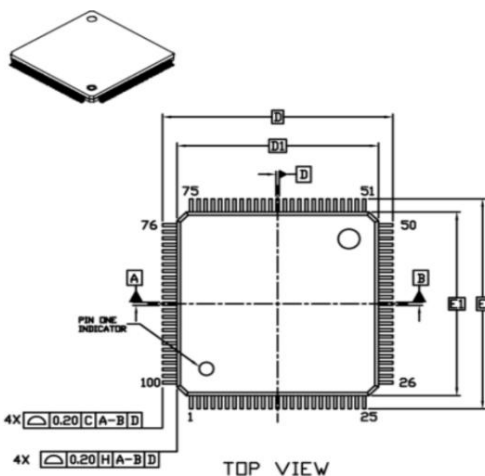
MECHANICAL CASE OUTLINE

ON Semiconductor®



TQFP100 14x14, 0.5P
CASE 932BR
ISSUE O

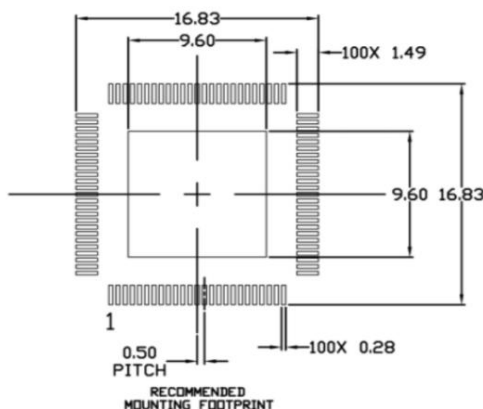
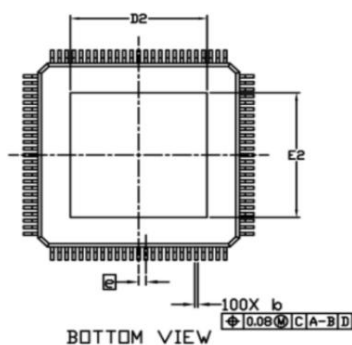
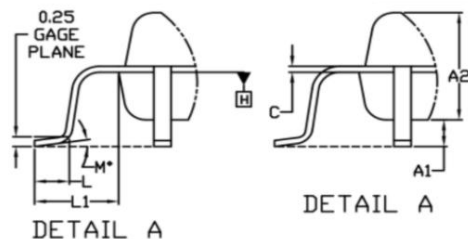
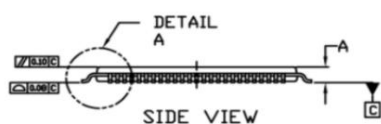
DATE 03 JUL 2018



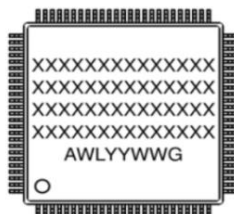
NOTES:

1. DIMENSIONING AND TOLERANCING PER ASME Y14.5M, 1994.
2. CONTROLLING DIMENSION- MILLIMETERS
3. DIMENSION B DOES NOT INCLUDE DAMBAR PROTRUSION. DAMBAR PROTRUSION SHALL BE 0.08 MAX. AT MMC. DAMBAR CANNOT BE LOCATED ON THE LOWER RADIUS OF THE FOOT.
4. DIMENSIONS D1 AND E1 DO NOT INCLUDE MOLD FLASH, PROTRUSIONS, OR GATE BURRS. MOLD FLASH, PROTRUSIONS, OR GATE BURRS SHALL NOT EXCEED .025 PER SIDE. DIMENSIONS D1 AND E1 ARE MAXIMUM PLASTIC SIZE. SEE DIMENSION J FOR MOLD FLASH.
5. THE TOP PACKAGE BODY SIZE IS SMALLER THAN THE BOTTOM PACKAGE SIZE AND TOP PACKAGE WILL NOT OVERHANG THE BOTTOM.
6. DATUM PLANE H IS LOCATED AT THE BOTTOM MOLD PARTING LINE COINCIDENT WITH WHERE THE LEAD EXITS THE BODY.
7. DIMENSIONS D1 AND E1 TO BE DETERMINED AT DATUM PLANE H.
8. DATUMS A-B AND C ARE DETERMINED AT DATUM PLANE H.
9. A1 IS DEFINED AS THE VERTICAL DISTANCE FROM THE SEATING PLANE TO THE LEAD CENTERLINE.
10. DIMENSIONS D AND E TO BE DETERMINED AT DATUM PLANE C.
11. EXPOSED PAD SIZE IS AFFECTED BY MOLD FLASH AND MOLD FLASH IS CONTROLLED BY FV/FINAL VISUAL INSPECTION SPEC.

DIM	MILLIMETERS		
	MIN.	NOM.	MAX.
A1	0.05	—	0.15
A2	0.95	1.00	1.05
c	0.17	0.22	0.27
D	15.90	16.00	16.20
D1	13.90	14.00	14.20
D2	9.50	REF	
E	15.90	16.00	16.20
E2	13.90	14.00	14.20
e	—	0.50	BSC
L	0.45	0.60	0.75
L1	—	1.00	REF
M	0.00	0.00	0.00



GENERIC MARKING DIAGRAM*



XXX = Specific Device Code
A = Assembly Location
WL = Wafer Lot
YY = Year
WW = Work Week
G = Pb-Free Package

*This information is generic. Please refer to device data sheet for actual part marking. Pb-Free indicator, "G" or microdot "•", may or may not be present. Some products may not follow the Generic Marking.

DOCUMENT NUMBER:	98AON94348G	Electronic versions are uncontrolled except when accessed directly from the Document Repository. Printed versions are uncontrolled except when stamped "CONTROLLED COPY" in red.
DESCRIPTION:	TQFP100 14X14, 0.5P	PAGE 1 OF 1

ON Semiconductor and  are trademarks of Semiconductor Components Industries, LLC dba ON Semiconductor or its subsidiaries in the United States and/or other countries. ON Semiconductor reserves the right to make changes without further notice to any products herein. ON Semiconductor makes no warranty, representation or guarantee regarding the suitability of its products for any particular purpose, nor does ON Semiconductor assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation special, consequential or incidental damages. ON Semiconductor does not convey any license under its patent rights nor the rights of others.

© Semiconductor Components Industries, LLC. 2018

www.onsemi.com

变更日志

日期	笔记
2021-05-05	首次公开发布。
2021年5月27日	新增Cog RAM、锁、CORDIC Solver和Smart I/O Pins部分。更新所有硬件连接图表。修正了 Hub RAM 大小类型,并澄清了Propeller 2 (P2X8C4M64P) RAM 内存配置中的地址类型桌子。
2021-07-09	将“其他文档和资源”更改为“前言”并描述了本文档的目的。添加了Cog Attention部分。在智能 I/O 引脚部分,修订的描述,包括方向和状态,引脚寄存器和特殊指令,增加了I/O 引脚时序部分,取代了Pin 模式表格增强了版本,并增强了所有 I/O 引脚电路以提高清晰度。澄清了CORDIC 求解器管道流和吞吐能力。增加了主机通信, P2监视器,塔克兹,并重启部分。
2022年11月1日	替换了I/O 引脚时序图并增强了相关描述。将内部快速振荡器速度从约 20 MHz 更新至 20+ MHz。修改了Cog RAM图表,并添加了有关调试中断调用/返回地址的注释。澄清了20 位地址“无关”位的声明。修复了第一个系统时钟表中VCO 的最大频率。修复了“锁使用”中的拼写错误。增强了CORDIC 求解器函数的命名。修正并增强了专用 microSD 启动文件的描述。

视差公司

视差公司
599 Menlo Drive, Suite 100 Rocklin,
CA 95765 美国

办公室 :+1 916-624-8333
美国免费电话 :888-512-1024

sales@parallax.com
support@parallax.com

www.parallax.com/p2
forums.parallax.com

购买 P2X8C4M64P 并不包含任何模拟其他设备或通过任何特定专有协议进行通信的许可;Parallax, Inc. 提供或引用的 P2X8C4M64P 连接对象和代码示例未获得许可,仅供研究和开发之用;最终用户必须获得协议许可持有者的许可,才能将许可协议用于其应用程序和产品。

Parallax, Inc. 不对其产品是否适用于任何特定用途作出任何保证、陈述或担保,也不承担因应用或使用任何产品而产生的任何责任,并特别声明不承担任何责任,包括但不限于间接或附带损害,即使 Parallax, Inc. 已被告知有此类损害的可能性。

版权所有 © 2022 Parallax, Inc. 保留所有权利。Parallax.Parallax 徽标、P2 徽标和 Propeller 是 Parallax, Inc. 的商标。